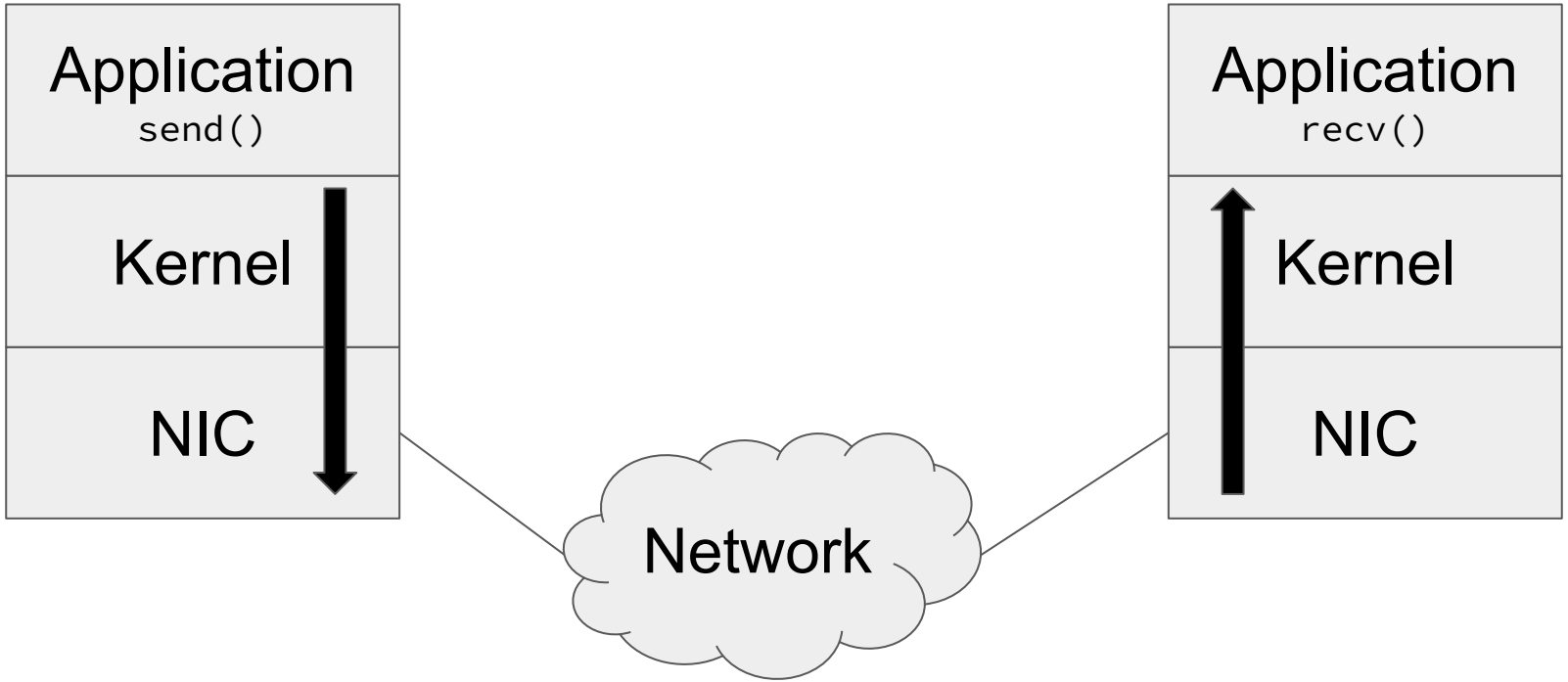
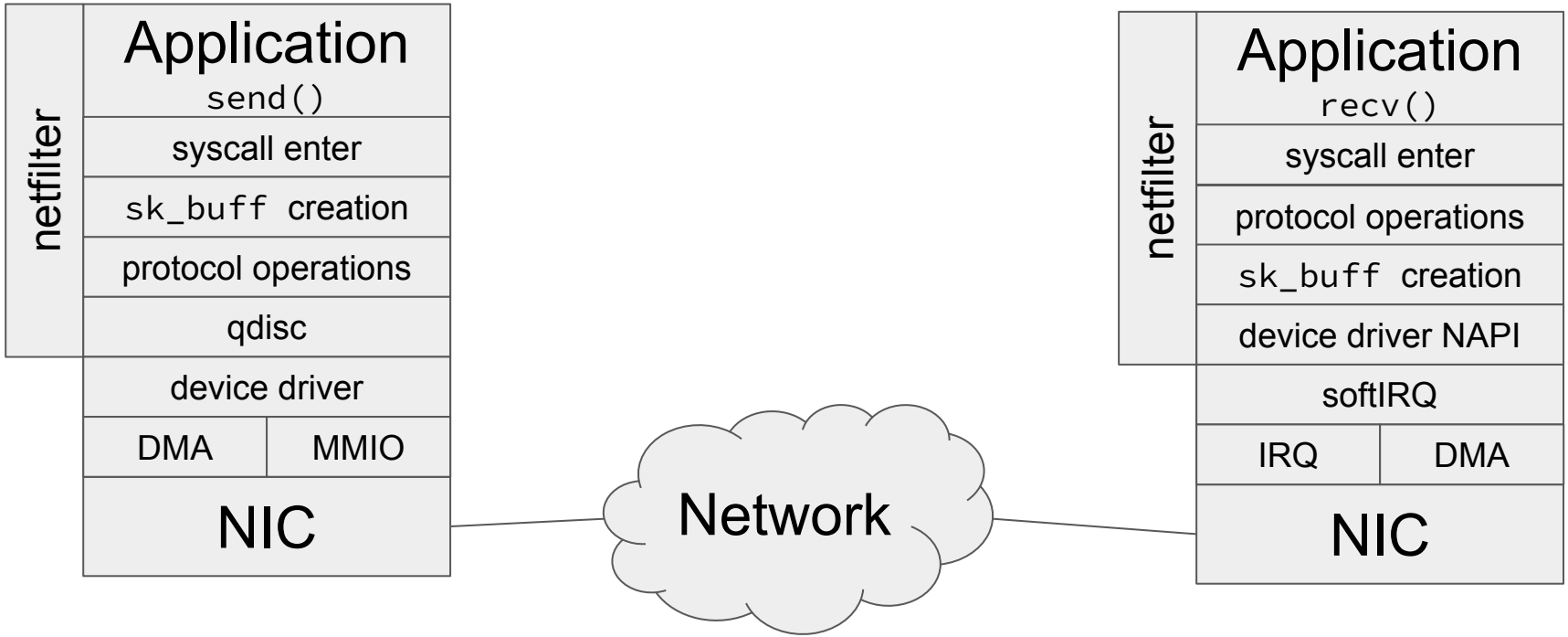
Linux Networking and (Infiniband RDMA)

Wonbin Jin

Network Stack Mental Model



Network Stack Mental Model



Goal

- Cover the general landscape of the networking infrastructure in Linux
- Broad overview of most parts, deep dive into some
- *Not about networking protocols*
- Infiniband

Goal

Trace the li eceive paths.

Ftrace: A ke

```
41 # tracer: function_graph
40 #
39 # CPU  DURATION                  FUNCTION CALLS
38 # |     |   |                     |   |   |   |
37  32)               |  __sys_sendto() {
36  32)               |    sockfd_lookup_light() {
35  32)   0.569 us    |      __fdget();
34  32)   1.538 us    |    }
33  32)               |    move_addr_to_kernel() {
32  32)               |      check_object_size() {
31  32)               |        object_size.part.0() {
30  32)   0.304 us    |          stack_object();
29  32)   1.077 us    |        }
28  32)   1.737 us    |      }
27  32)   2.565 us    |    }
26  32)               |    security_socket_sendmsg() {
25  32)               |      apparmor_socket_sendmsg() {
24  32)               |        aa_inet_msg_perm() {
23  32)   0.480 us    |          __cond_resched();
22  32)   1.163 us    |        }
21  32)   1.678 us    |      }
20  32)   2.478 us    |    }
```

Linux 6.8.0

Transmit Path: Entry

`__sys_sendto`: Entrypoint into the kernel

`sockfd_lo` and get the associated s

```
read(filefd, buf, sizeof(buf))

read(sockfd, buf, sizeof(buf))
```

`inet_sendmsg`: Dispatch to the corresponding send function depending on the socket protocol

Transmit Path: Some Optimizations

Indirect call wrapper macro

```
if (sk->sk_prot->sendmsg == tcp_sendmsg)
    return tcp_sendmsg(sk, msg, size);
else if (sk->sk_prot->sendmsg == udp_sendmsg)
    return udp_sendmsg(sk, msg, size);
else
    return sk->sk_prot->sendmsg(sk, msg, size);
```

Directly calling the function instead of through the function pointer.

More compiler optimization opportunities

Transmit Path: Some Optimizations

`likely`, `unlikely`

```
if (likely(...)) {
}

if (unlikely(...)) {
}
```

Macro around `__builtin_expect`

More optimal code layout.

Transmit Path: Packet construction

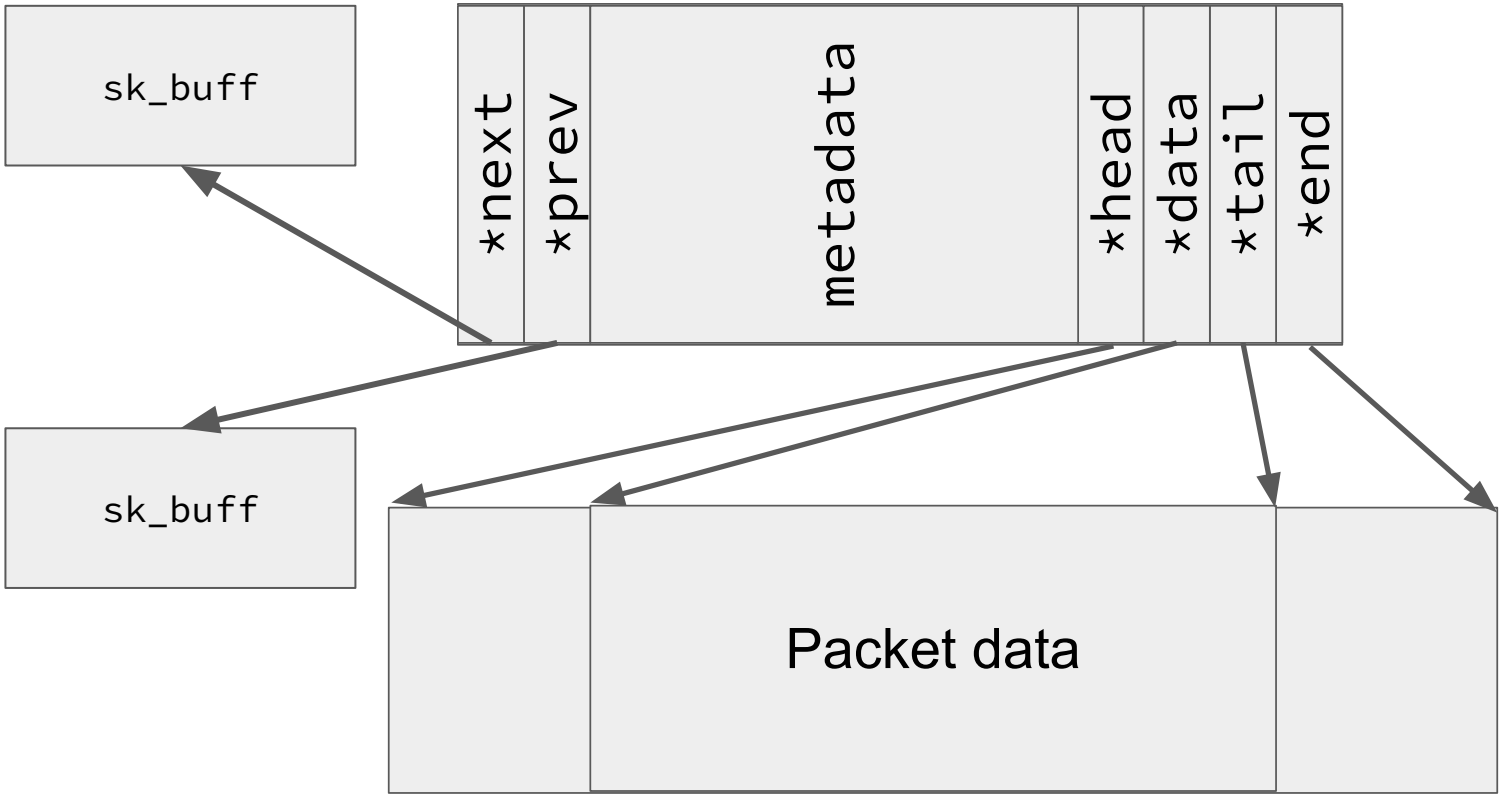
`udp/tcp_sendmsg`: Protocol specific send function

`ip_route_output_flow`: Routing table lookup to determine which network device and next hop to send to

`ip_make_skb`: Construct the `sk_buff` packet structure

`ip_append_data`: Typically copies data from user space

sk_buff: Packet representation in the kernel



Transmit Path: Populating headers

`udp_send_skb`: Populate UDP header, UDP specific operations

`ip_send_skb`: Run netfilter hooks

`ip_finish_output`: Finish populating IP header, more netfilter hooks

Netfilter hooks

Hooks executed in the kernel at specific points inspecting, modifying, dropping packets

Powers `iptables`, firewalls, etc

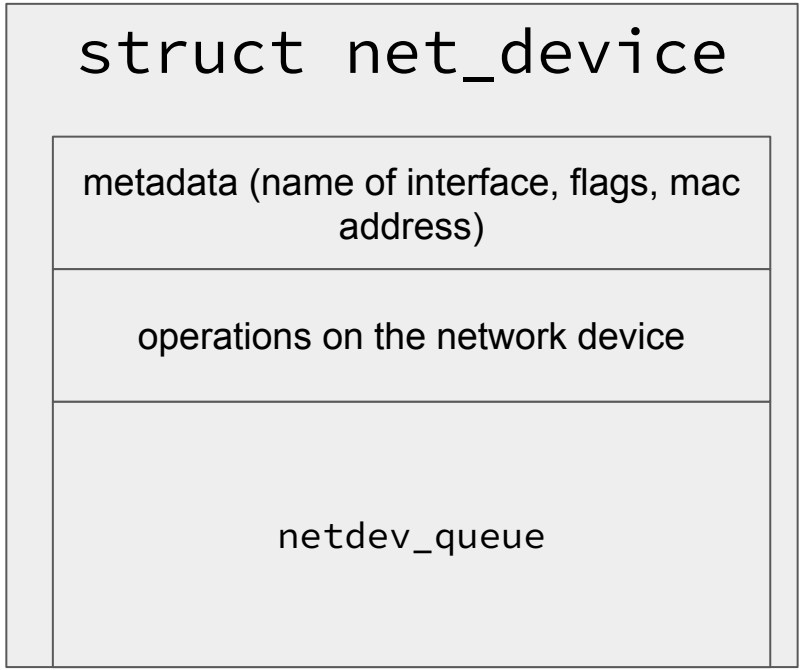
Transmit Path: Queuing

`ip_neigh_for_gw`: Neighbor discovery protocol operations

`dev_queue_xmit`: Queue packet on a `net_device` for transmission

net_device, qdisc

struct that represents a physical network device.



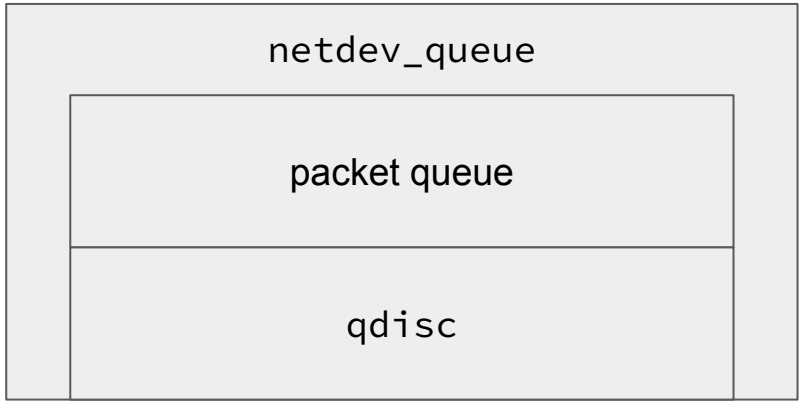
net_device, qdisc

qdisc is the mechanism for packet queueing policies, it operates on the transmit queue

Implements enqueue, dequeue operations

Usually fair queue controlled delay

Controlled with tc

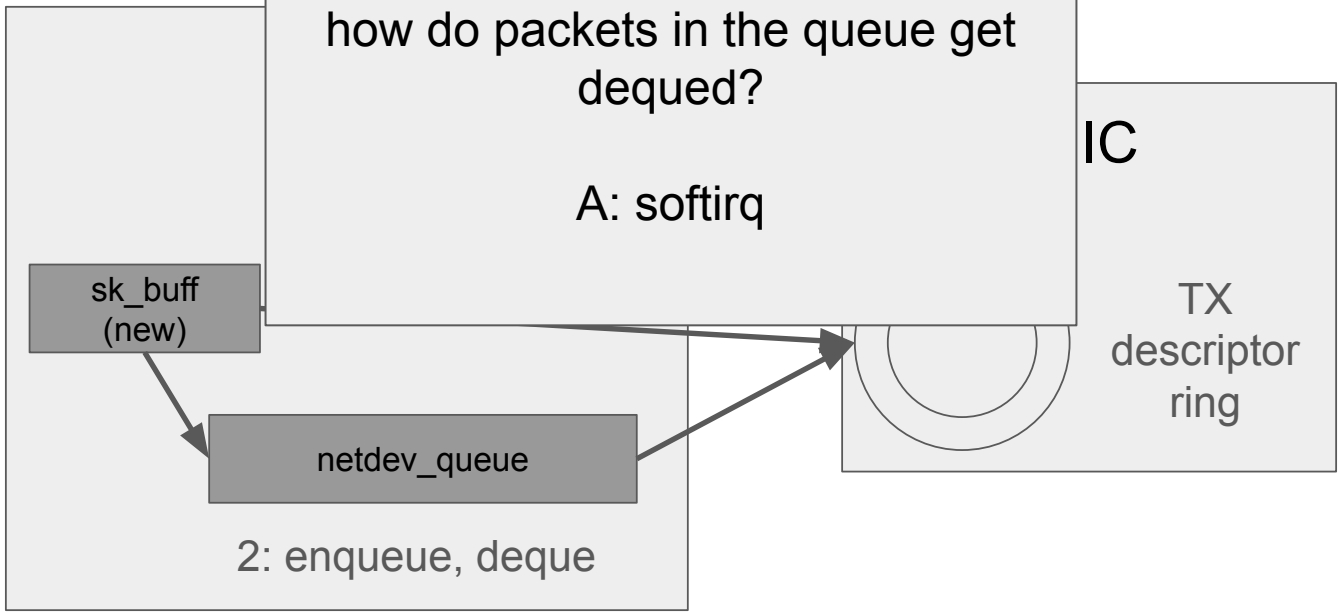


net_device, qdisc

Sometimes you can skip the queue, other times the packet that you just crea end up transmitting a dif

Q: If no new packets are being sent, how do packets in the queue get dequed?

A: softirq



Transmit Path: Network Device Drivers

`dev_hard_start_xmit`: Handoff between generic network stack to network device driver

`ndo_start_xmit`: Network device driver entrypoint

Network Device Drivers

Loadable kernel modules that implements the interface between the generic kernel networking stack and a specific network device.

Device drivers register callbacks in the `net_device_ops` struct

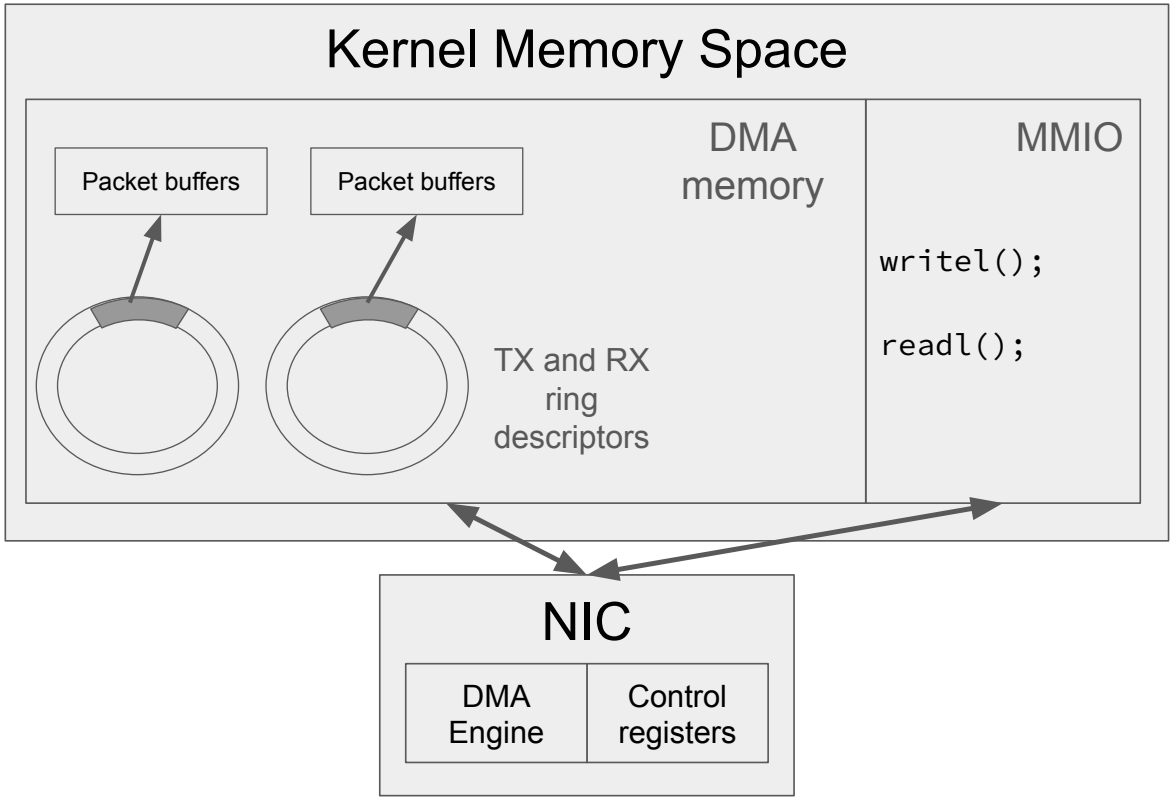
```
struct net_device_ops {
    int (*ndo_open)(struct net_device *);
    netdev_tx_t (*ndo_start_xmit)(struct sk_buff *, struct net_device *);
};
```

Transmit Path: Network Device Drivers

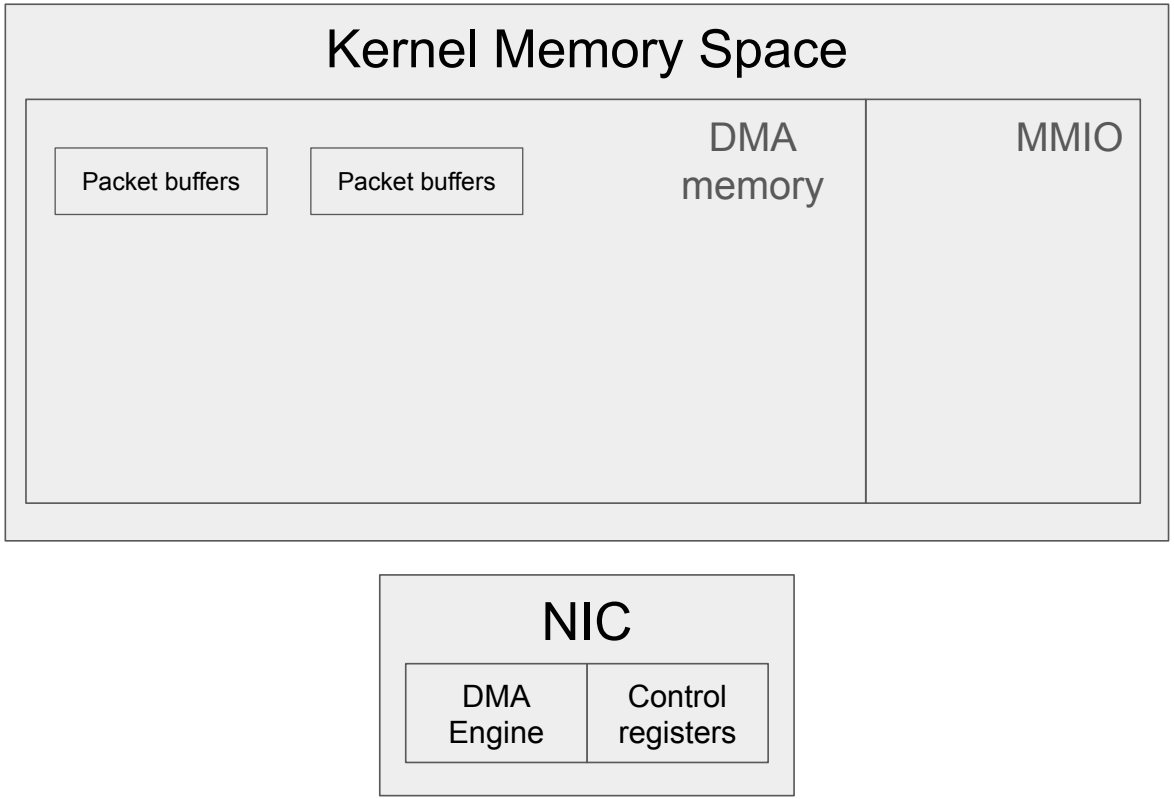
`igb_xmit_frame`: igb is the Intel device driver for Ethernet Network Connection

```
wbjin@node0:/usr/src/linux-source-6.8.0$ lspci | grep Ethernet
01:00.0 Ethernet controller: Intel Corporation I350 Gigabit Network Connection (rev 01)
01:00.1 Ethernet controller: Intel Corporation I350 Gigabit Network Connection (rev 01)
06:00.0 Ethernet controller: Intel Corporation 82599ES 10-Gigabit SFI/SFP+ Network Connection (rev 01)
06:00.1 Ethernet controller: Intel Corporation 82599ES 10-Gigabit SFI/SFP+ Network Connection (rev 01)
```

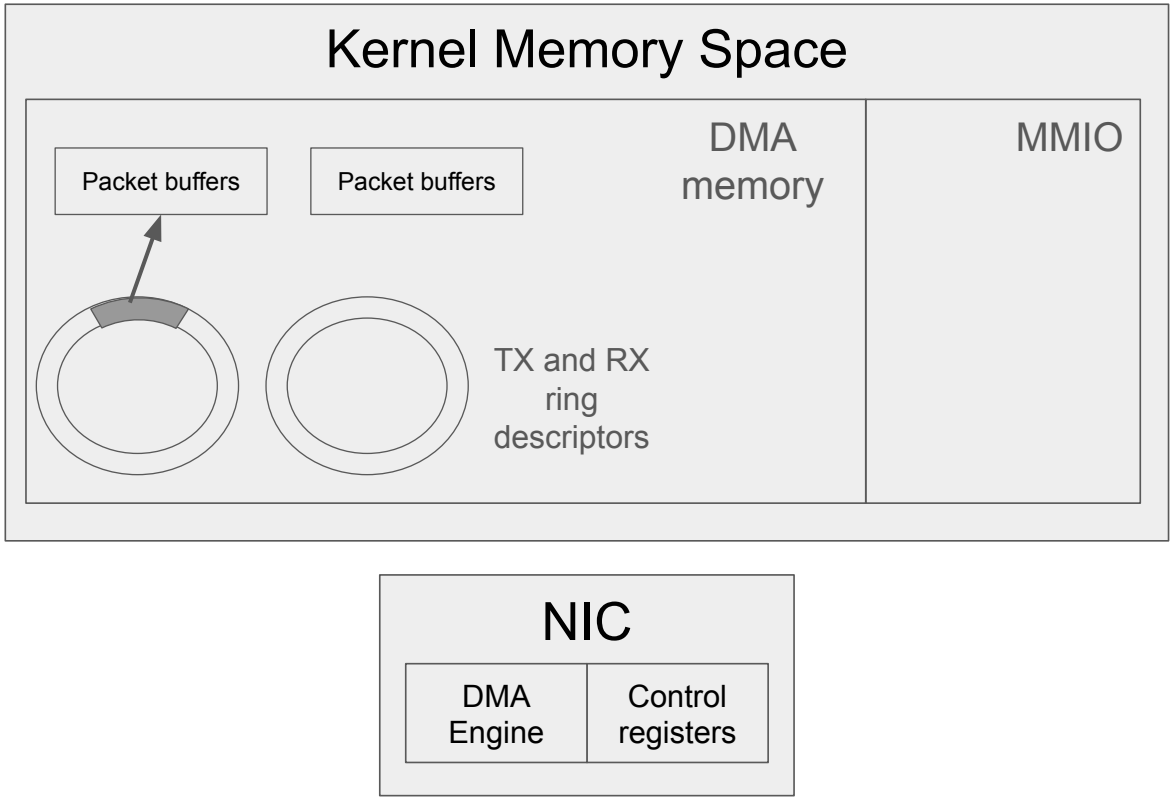
Transmit Path: Network Device Drivers



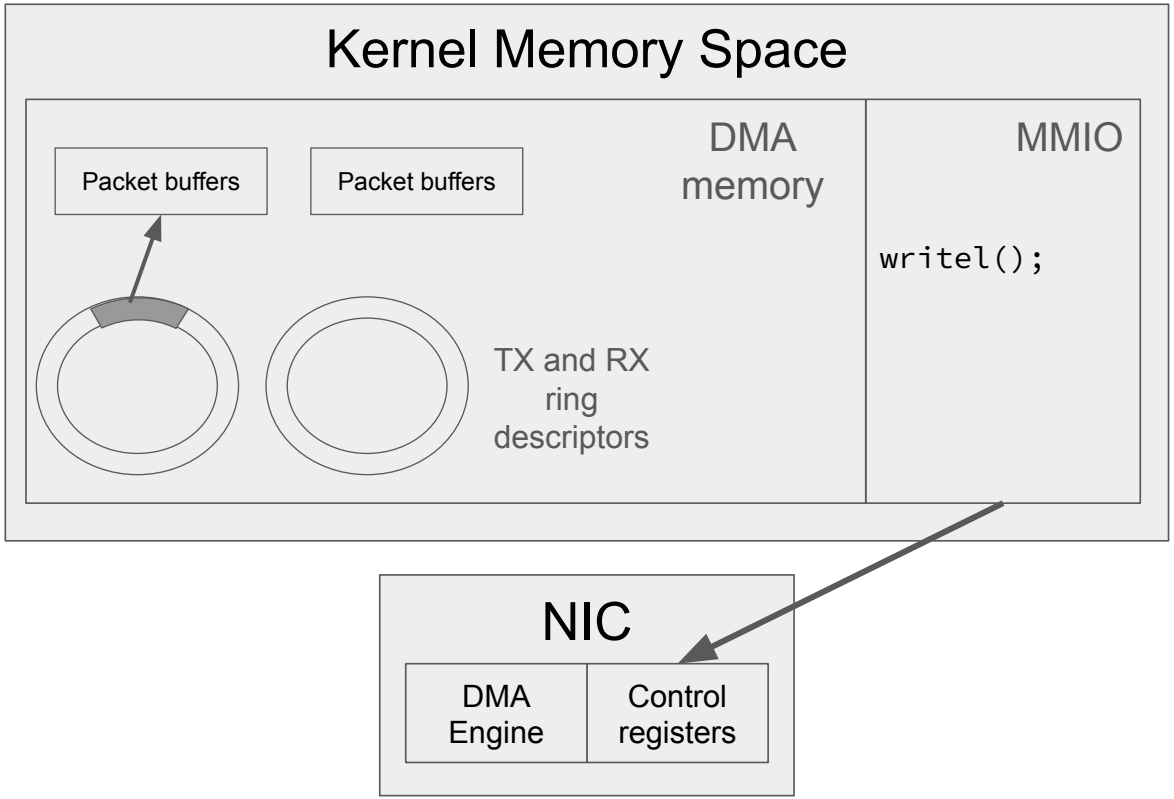
1. Map packets to DMA



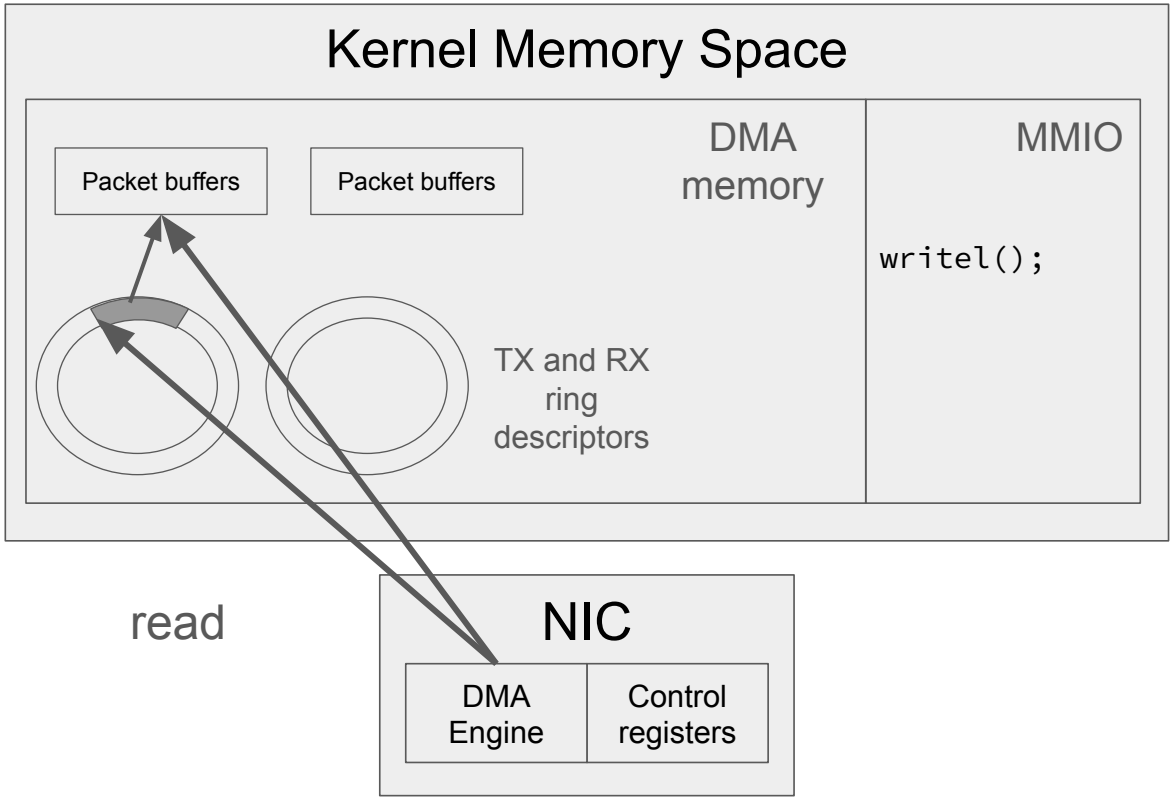
2. Write address of packet buffer into ring descriptor



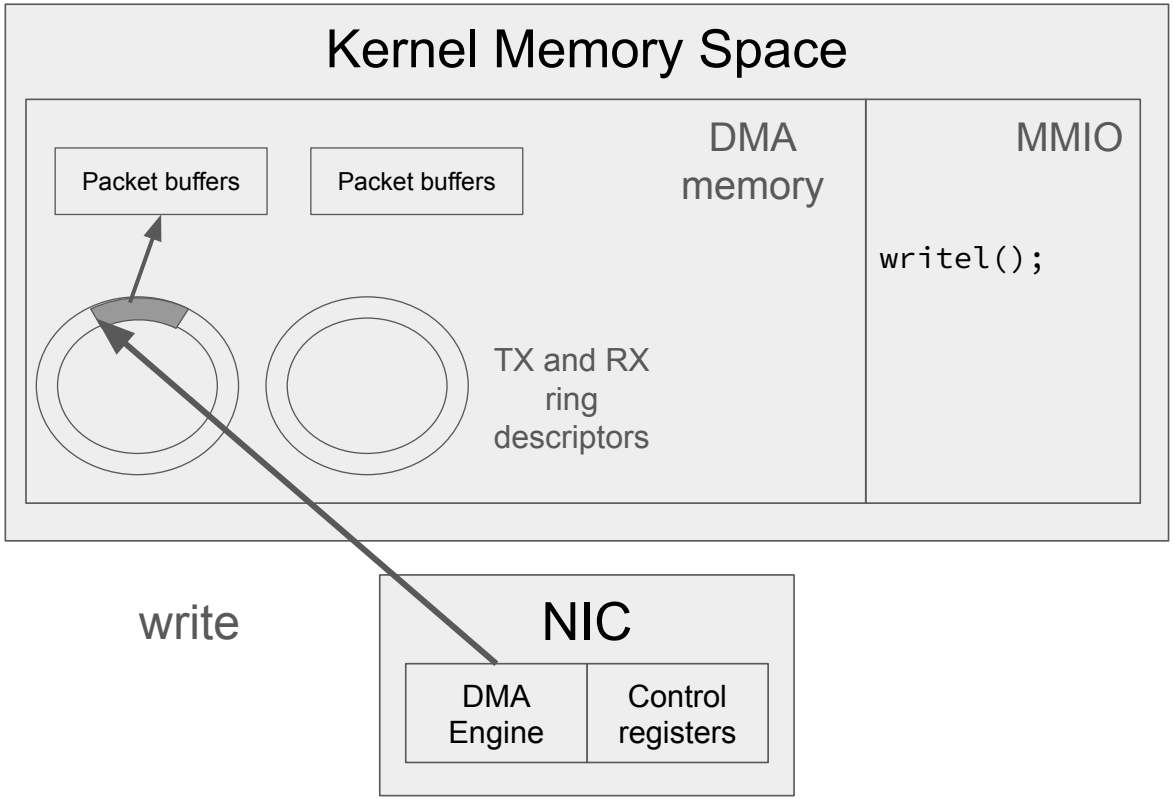
3. Update tail index on the TX ring buffer



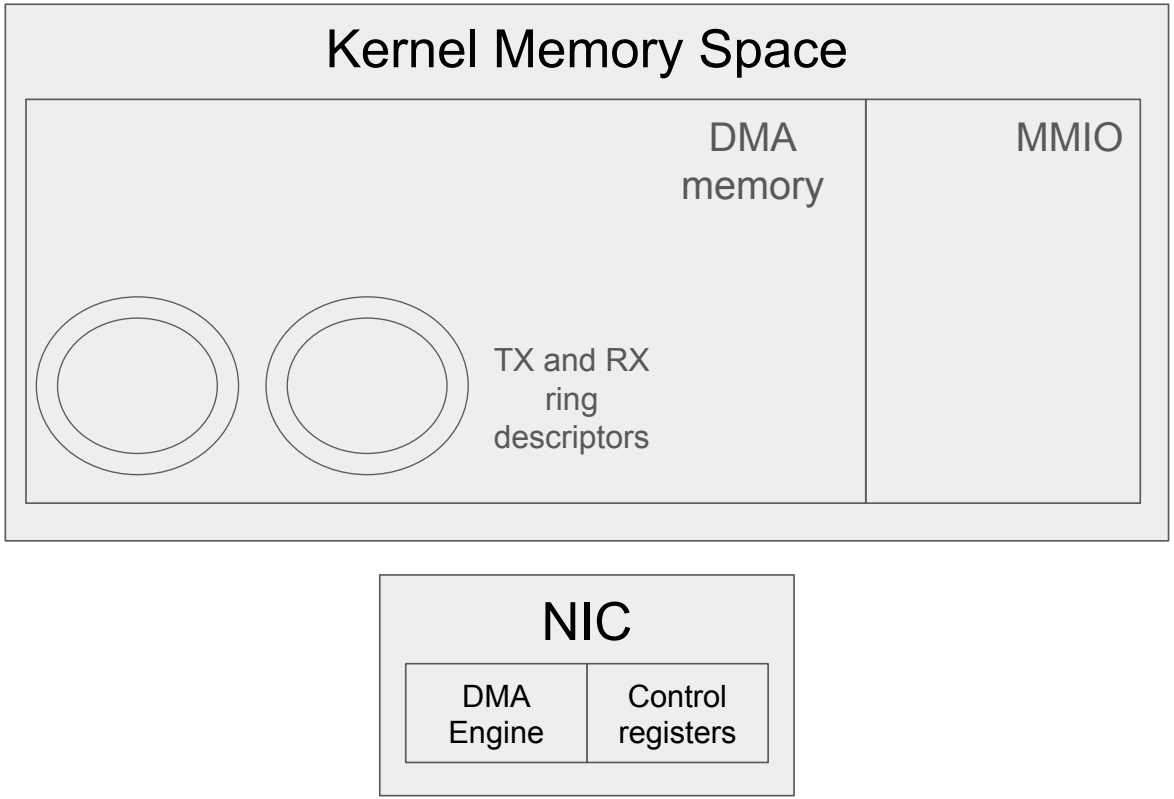
4. Nic reads TX ring and packet buffer with DMA



4. Nic transmits the packet, marks descriptor as processed



5. Kernel frees packet buffers



Transmit Path: Return

```
39) + 47.606 us   |                }
39) + 50.380 us   |              }
39) + 52.171 us   |            }
39) + 53.634 us   |          }
39)   0.258 us    |          dst_release();
39) + 96.073 us   |        }
39) ! 112.378 us  |      }
39) ! 121.848 us  |    }
```

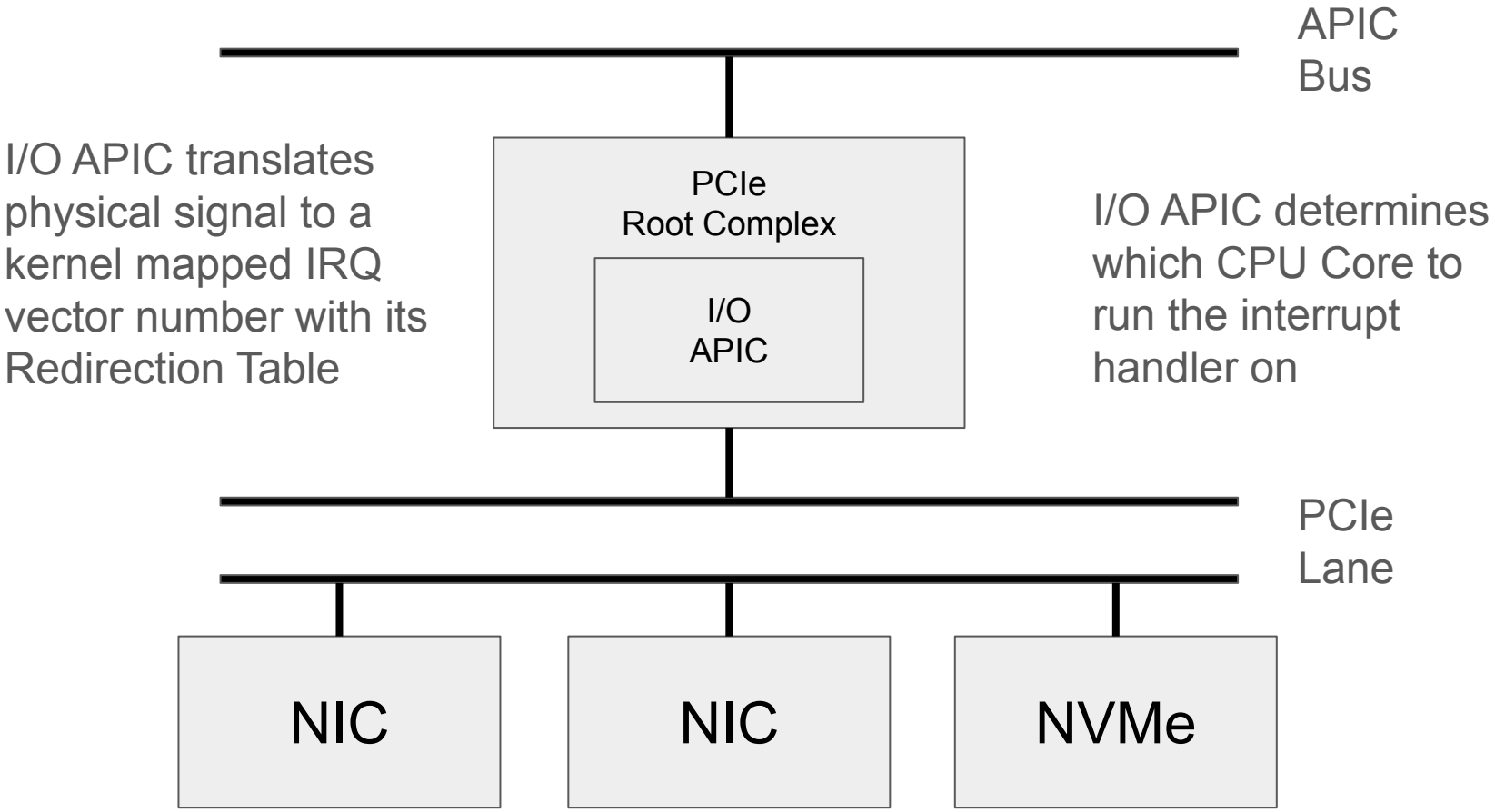
Receive Path: Interrupts

Signal from a peripheral device to the CPU indicating an event on the device.

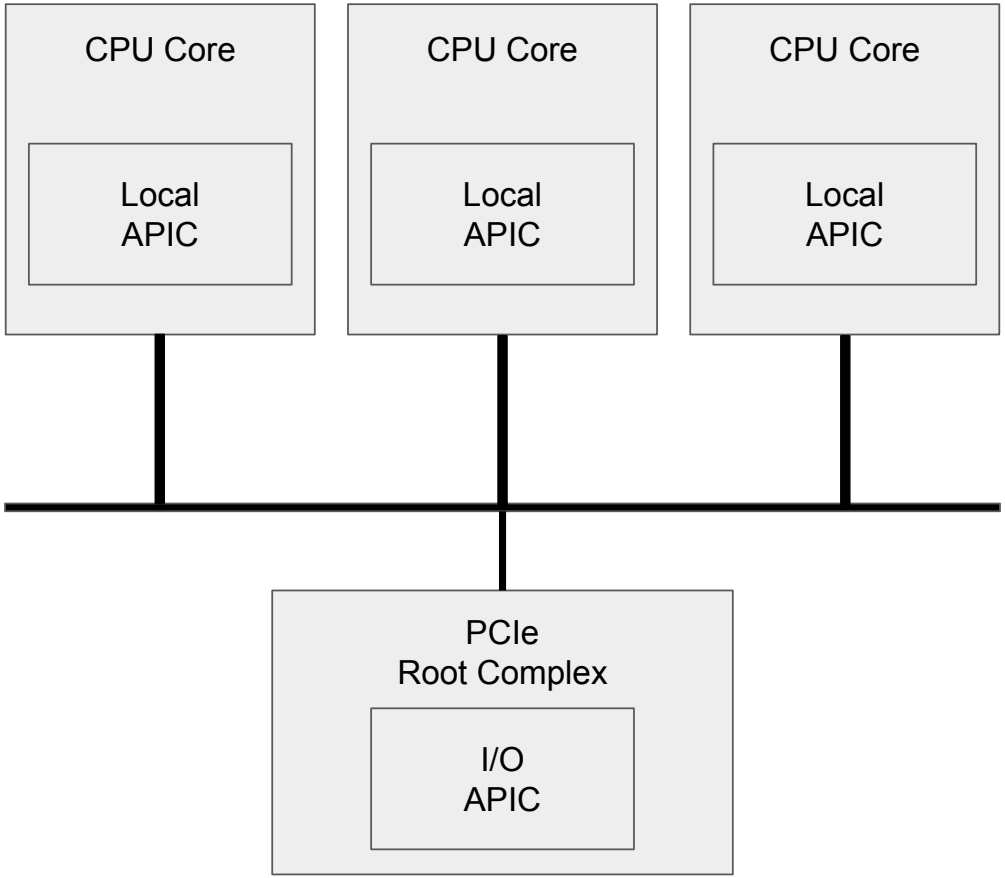
Linux refers to **hardware** interrupts as Interrupt ReQuest (IRQ)

Linux distinguishes between hardware and software IRQs

Receive Path: NIC Interrupt



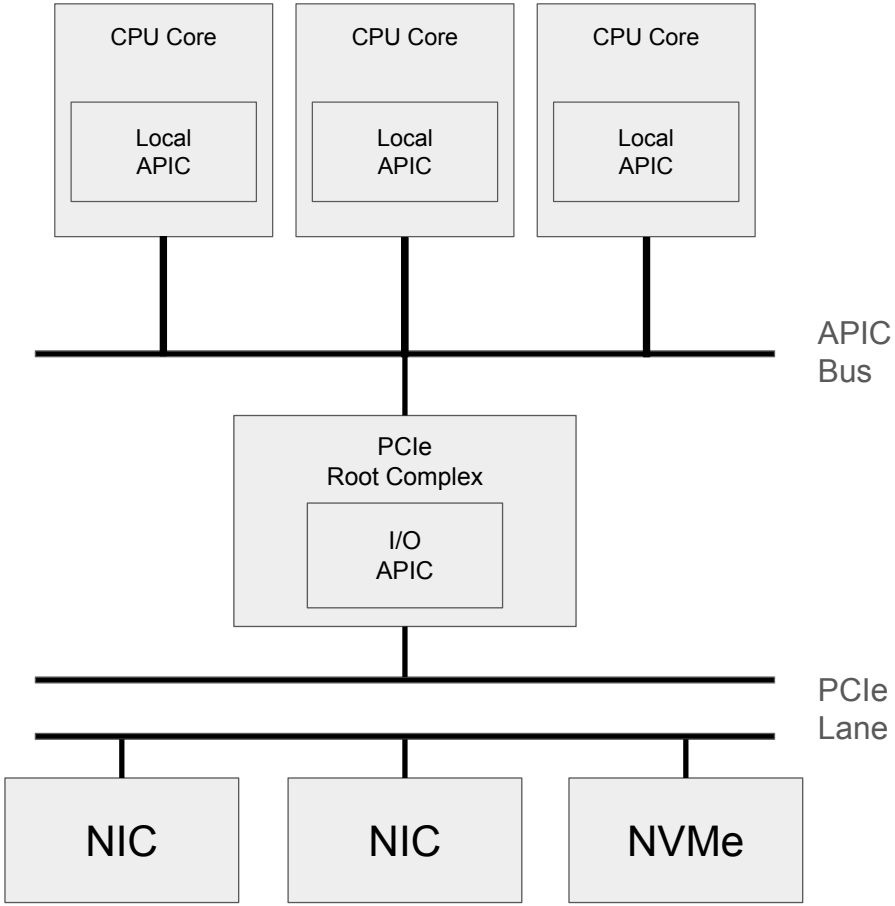
Receive Path: NIC Interrupt



Local APIC listens to the bus for signals from the I/O APIC, it signals the CPU core to take an interrupt

APIC Bus

Receive Path: NIC Interrupt



MSI-X makes it possible to bypass the I/O APIC.

NIC issues PCIe message write to local APIC directly.

Receive Path: Interrupt Service Routine

`handle_irq_event`: Entry point into interrupt handler. Dispatches to the corresponding handler associated with the interrupt vector

`igb_msix_ring`: The interrupt handler for igb device driver.

```
static irqreturn_t igb_msix_ring(int irq, void *data)
{
        struct igb_q_vector *q_vector = data;

        /* Write the ITR value calculated from the previous interrupt. */
        igb_write_itr(q_vector);

        napi_schedule(&q_vector->napi);

        return IRQ_HANDLED;
}
```

Receive Path: Interrupt Service Routine

Notice that there wasn’t any processing of data, network protocol functionality, etc.

Linux divides ISRs into top and bottom halves.

Top half is simple, fast, and primarily acknowledges the interrupt.

Bottom half is where the data processing and network protocol operations happen.

Receive Path: Softirq

```
wbjin@node0:~$ ps aux | grep kworker
root           4  0.0  0.0      0     0 ?        I<   Mar12   0:00 [kworker/R-rcu_g]
root           5  0.0  0.0      0     0 ?        I<   Mar12   0:00 [kworker/R-rcu_p]
root           6  0.0  0.0      0     0 ?        I<   Mar12   0:00 [kworker/R-slub_]
root           7  0.0  0.0      0     0 ?        I<   Mar12   0:00 [kworker/R-netns]
root          10  0.0  0.0      0     0 ?        I<   Mar12   0:00 [kworker/0:0H-events_highpri]
root          11  0.0  0.0      0     0 ?        I    Mar12   0:08 [kworker/u80:0-ipv6_addrconf]
root          12  0.0  0.0      0     0 ?        I<   Mar12   0:00 [kworker/R-mm_pe]
root          26  0.0  0.0      0     0 ?        I<   Mar12   0:00 [kworker/1:0H-events_highpri]
root          32  0.0  0.0      0     0 ?        I<   Mar12   0:00 [kworker/2:0H-events_highpri]
root          38  0.0  0.0      0     0 ?        I<   Mar12   0:00 [kworker/3:0H-events_highpri]
root          44  0.0  0.0      0     0 ?        I<   Mar12   0:00 [kworker/4:0H-events_highpri]
root          50  0.0  0.0      0     0 ?        I<   Mar12   0:00 [kworker/5:0H-events_highpri]
root          56  0.0  0.0      0     0 ?        I<   Mar12   0:00 [kworker/6:0H-events_highpri]
root          62  0.0  0.0      0     0 ?        I<   Mar12   0:00 [kworker/7:0H-events_highpri]
root          68  0.0  0.0      0     0 ?        I<   Mar12   0:00 [kworker/8:0H-events_highpri]
root          74  0.0  0.0      0     0 ?        I<   Mar12   0:00 [kworker/9:0H-events_highpri]
root          79  0.0  0.0      0     0 ?        I    Mar12   0:10 [kworker/10:0-mm_percpu_wq]
```

```
root         132  0.0  0.0      0     0 ?        S    Mar12   0:00 [ksoftirqd/19]
```

Receive Path: New API (NAPI)

An interrupt per packet is inefficient and causes interrupt flooding where the CPU spends all of its time handling interrupts.

NAPI changes interrupt driven to poll driven mechanism

First interrupt disables NIC RX interrupts and enables NAPI polling

Receive Path: SoftIRQ + NAPI

`net_rx_action`: Entrypoint into the network stack bottom half ISR, run as a softIRQ

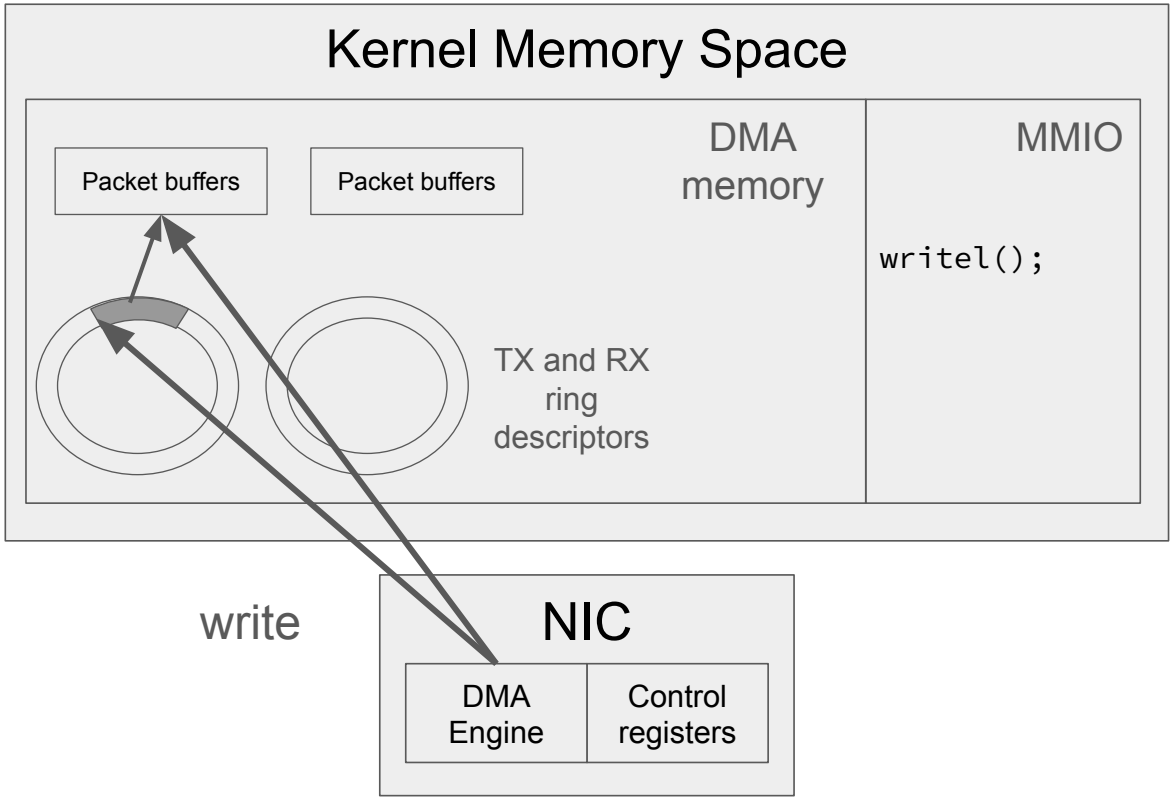
`napi_poll`: Dispatch to the corresponding device driver polling functionality

Receive Path: SoftIRQ + NAPI

`igb_clean_tx_irq`: Reclaim `sk_buff` structure for transmitted packets, clear corresponding ring descriptors

`igb_clean_rx_irq`: The main driver of the receive functionality. Runs XDP if enabled. Allocates and constructs sk_buff structure. Hands off to Generic Receive Offload (GRO). Reclaim `sk_buff` structures for processed packets and corresponding ring descriptors

Receive Path: SoftIRQ + NAPI



Receive Path: Network Operations

`napi_complete_done`: Finish polling and re-enable interrupts.

`ip_rcv_core`: IP header validation

`ip_rcv_finish_core`: Make routing decision. For end host, destination is local.

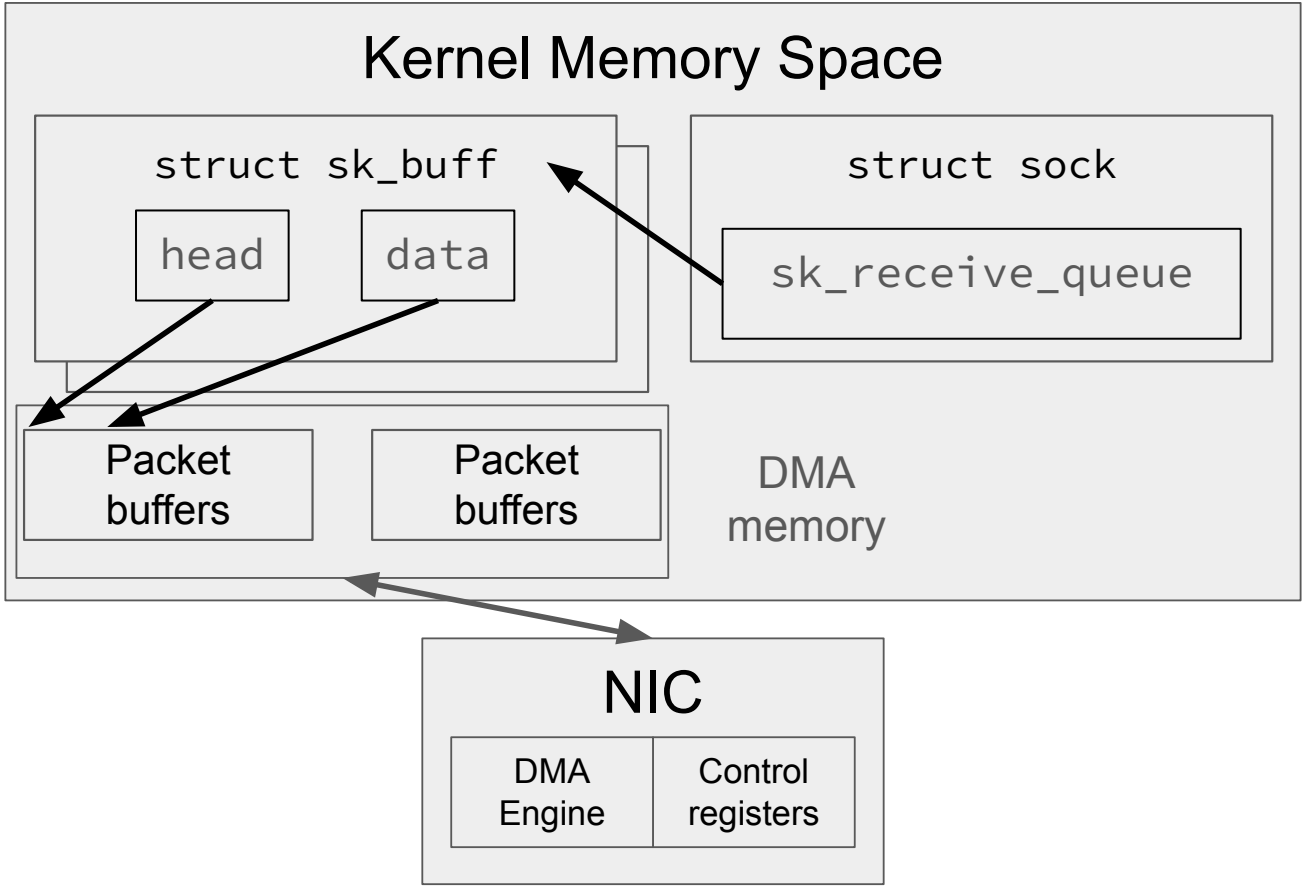
Receive Path: Network Operations

`udp_rcv`: UDP header check and find the associated socket.

`__udp_enqueue_schedule_skb`: Enqueue the packet into the socket's receive queue. `sk_buff` now "owned" by the socket.

`sock_def_readable`: Mark the socket as ready to be read from. Awake processes waiting on the socket.

Receive Path: After SoftIRQ, NAPI



Receive Path: After SoftIRQ, NAPI

```
10   2) + 56.783 us   |                    }
 9   2) + 81.792 us   |                  }
 8   2) + 83.768 us   |                }
 7   2) + 86.273 us   |              }
 6   2)   0.485 us    |              __rcu_read_unlock();
 5   2) + 90.945 us   |            }
 4   2) + 91.908 us   |          }
 3   2)   0.646 us    |          igb_ring_irq_enable [igb]();
 2   2) ! 116.551 us  |        }
 1   2) ! 117.506 us  |      }
212  2) ! 127.119 us  |    }
```

Receive Path: Entry

`__sys_recvfrom`: Entrypoint into the kernel

`sockfd_lookup_light`, `sock_recvmsg`, `inet_recvmsg`, `udp_recvmsg`: Similar to the send path.

Receive Path: Waiting for a packet

`__skb_recv_udp`: Main logic of receiving a UDP datagram.

`__skb_try_recv_from_queue`: Tries to dequeue a packet from the `socket` packet queue

`__skb_wait_for_more_packets`: Adds the process to the `socket's` wait queue

Receive Path: Waiting for a packet

`schedule`: Allows another process to run.

`__skb_try_recv_from_queue`: Tries to dequeue a packet from the `socket` packet queue, actually gets a packet this time.

Receive Path: Copy to user buffer

`__check_object_size`: Validates the user buffer. Check that the passed in buffer is in user space.

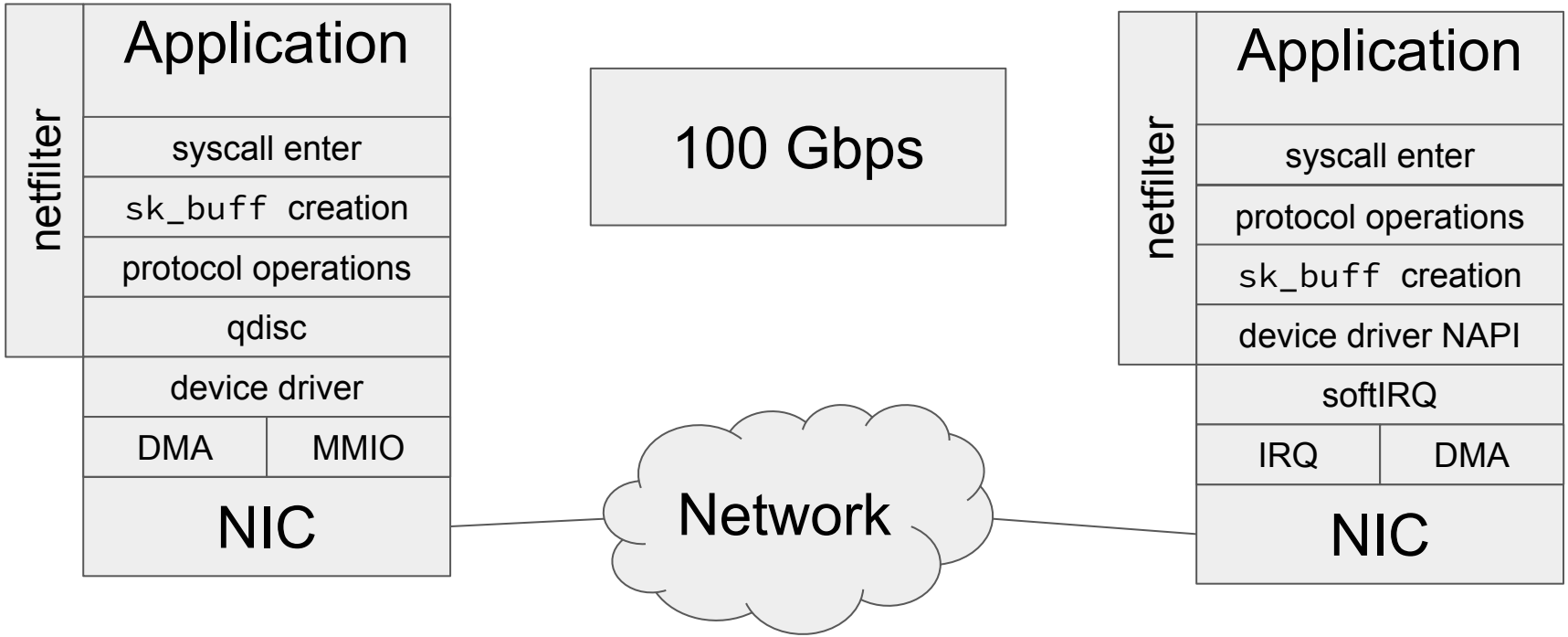
`copy_to_user`: Copy packet payload to the user buffer.

`skb_consume_udp`: Free `sk_buff`

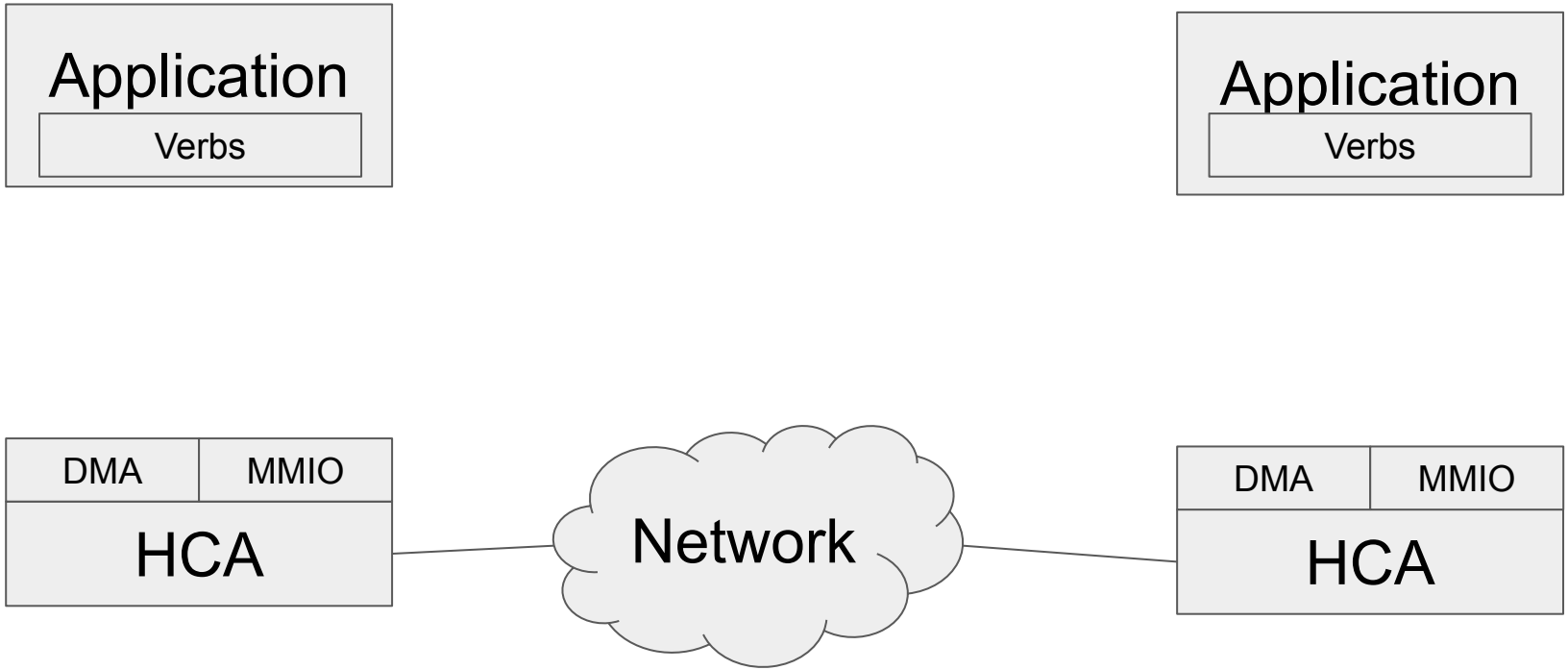
Receive Path

```
 7  17)    3.170 us    |                }
 6  17)    4.077 us    |              }
 5  17)    8.069 us    |            }
 4  17)    9.012 us    |          }
 3  17) $ 5949997 us   |        }
 2  17) $ 5949998 us   |      }
 1  17) $ 5950003 us   |    }
184  17) $ 5950021 us   |  }
```

What would you remove?



What would you remove?



Infiniband RDMA

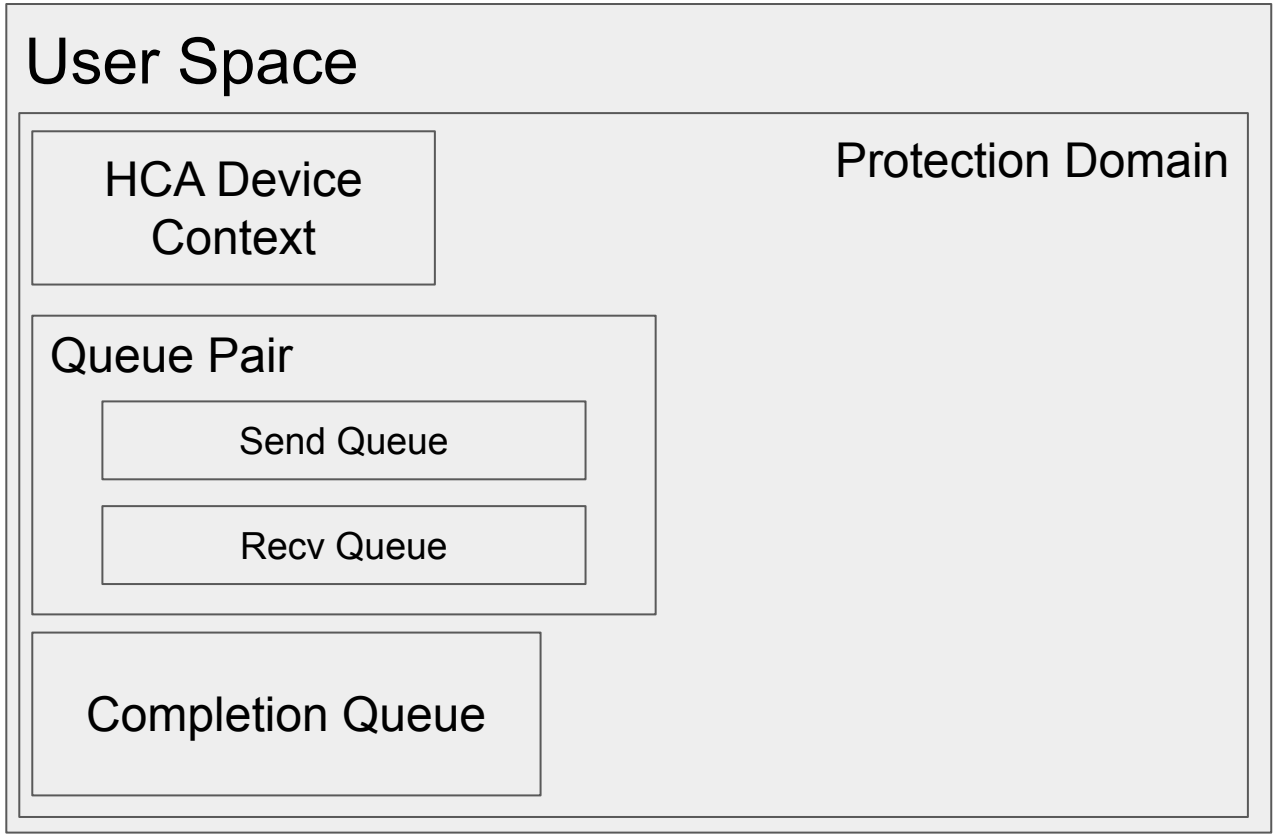
https://insujang.github.io/2020-02-09/introduction-to-programming-infiniband/

1. Verbs programming model and `libibverbs`

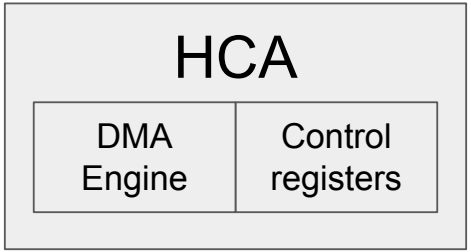
Fundamentally different programming model from BSD sockets

With sockets, you ask the kernel to send and receive.
With Verbs, you tell the HCA what to send and receive.

1. Verbs programming model and `libibverbs`



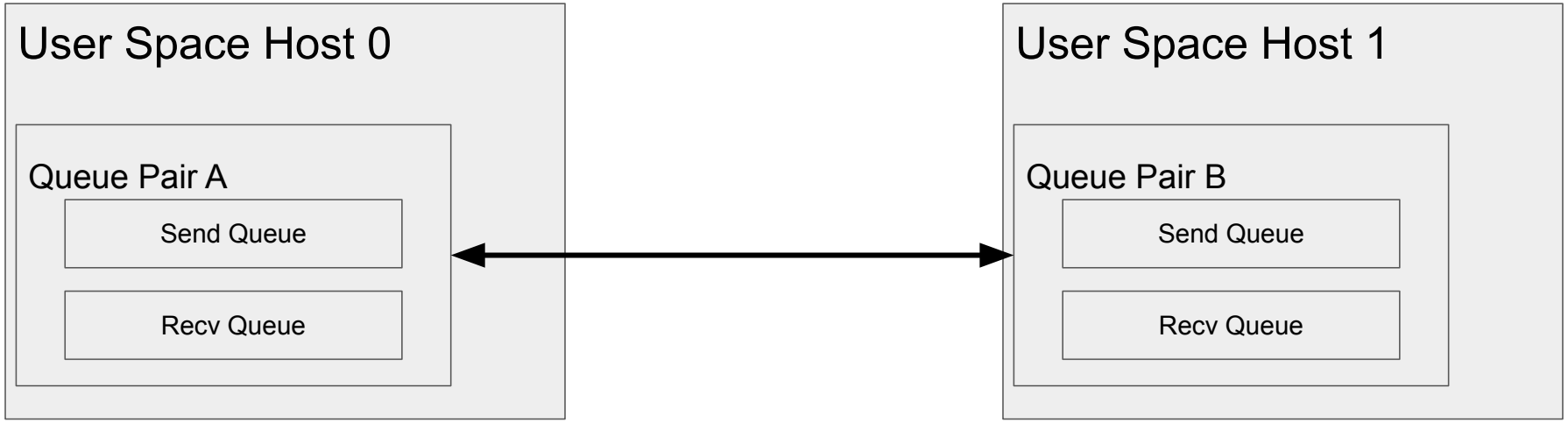

```
ibv_create_qp
```



2. Exchange Identifier Information

Exchange

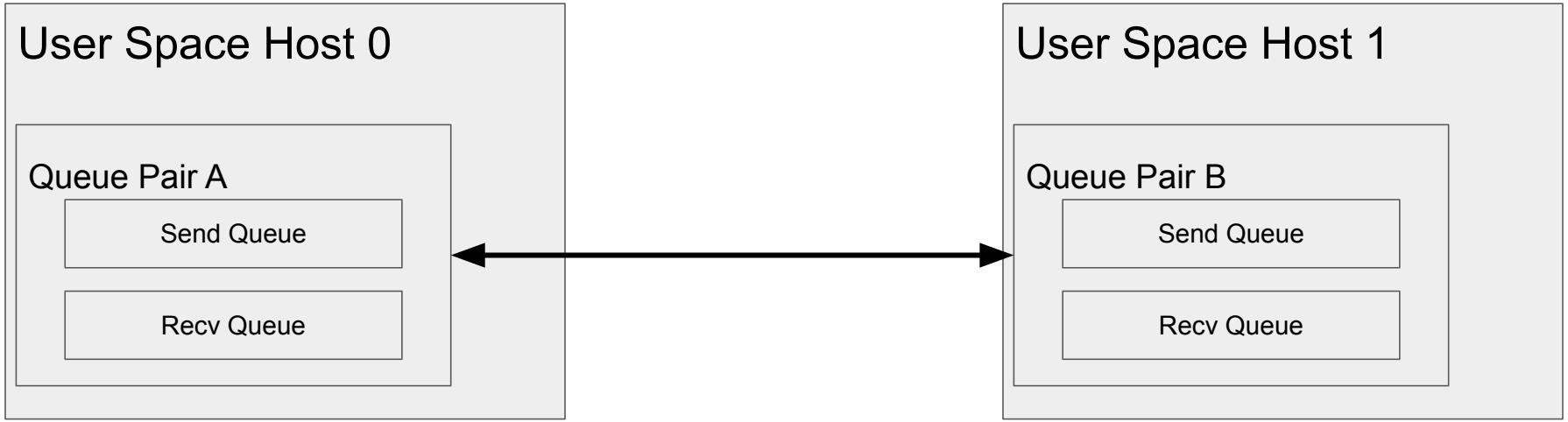
- Queue pair number
- Infiniband local identifier
- Initial packet sequence number



3. Update queue pair state

1. RESET
2. INIT: Initialize QP locally
3. RTR: Tie QP with a remote peer, able to receive
4. RTS: Able to send

```
ibv_modify_qp
```



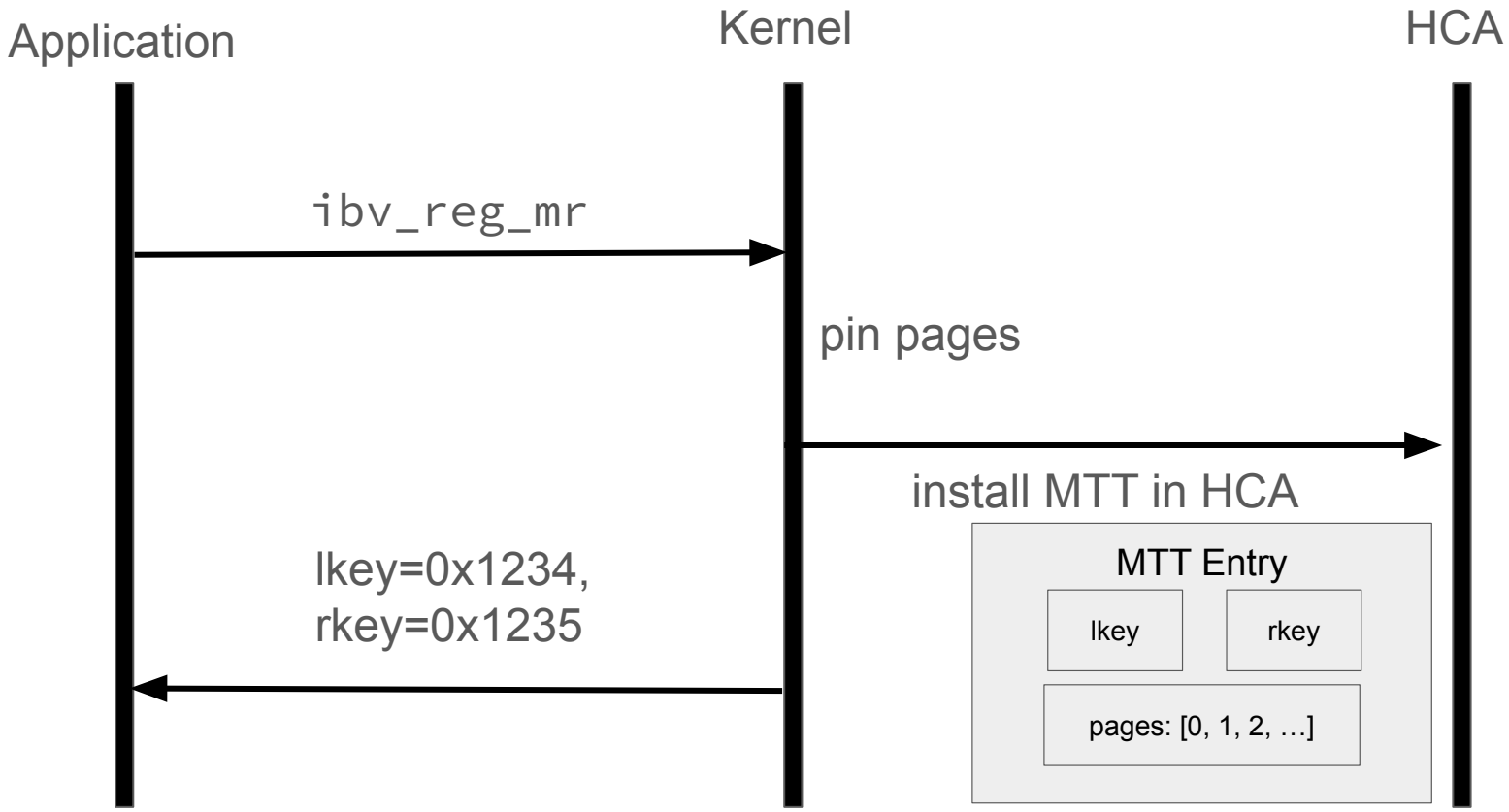
4. Memory Registration

Need to expose user space virtual address to hardware

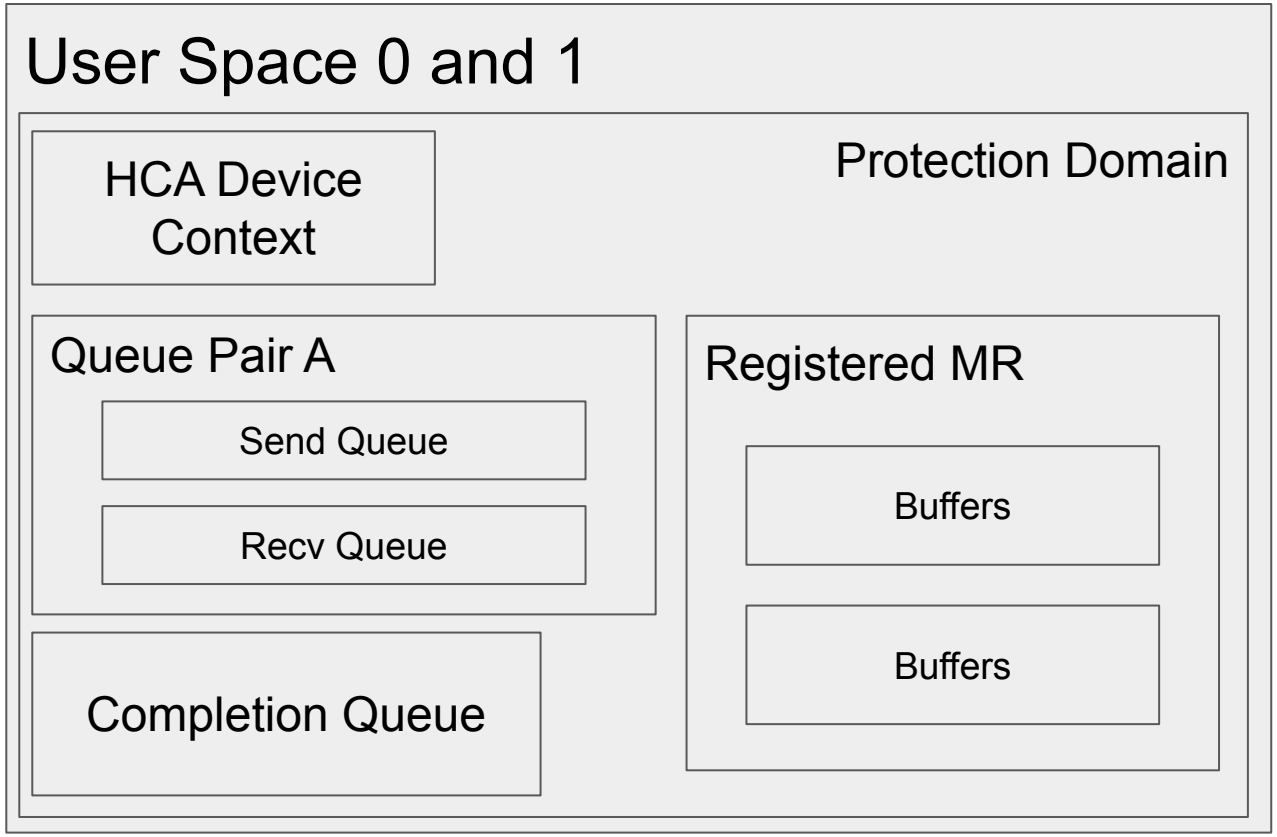
Need to stop the DMA mapped pages from being swapped

Need to constrain HCA so it can’t read write to anywhere in memory

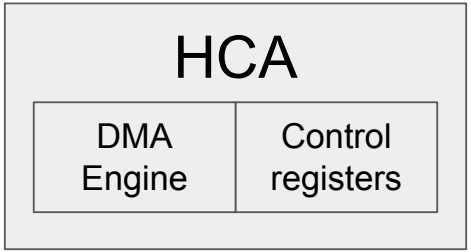
4. Memory Registration



4. Memory Registration




```
ibv_reg_mr
```



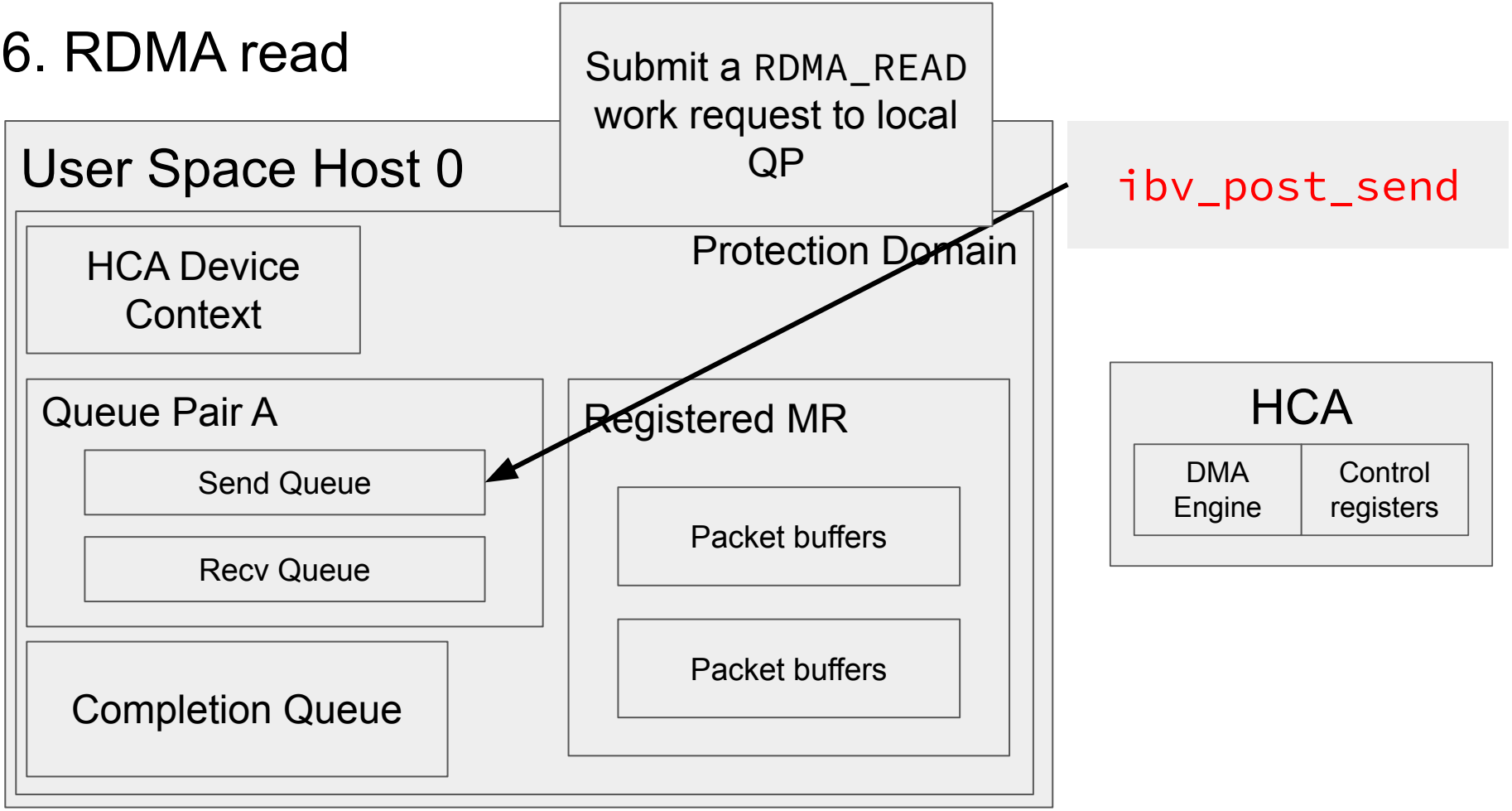
5. Share memory registration information

The remote key (rkey) is needed for remote hosts to issue RDMA.

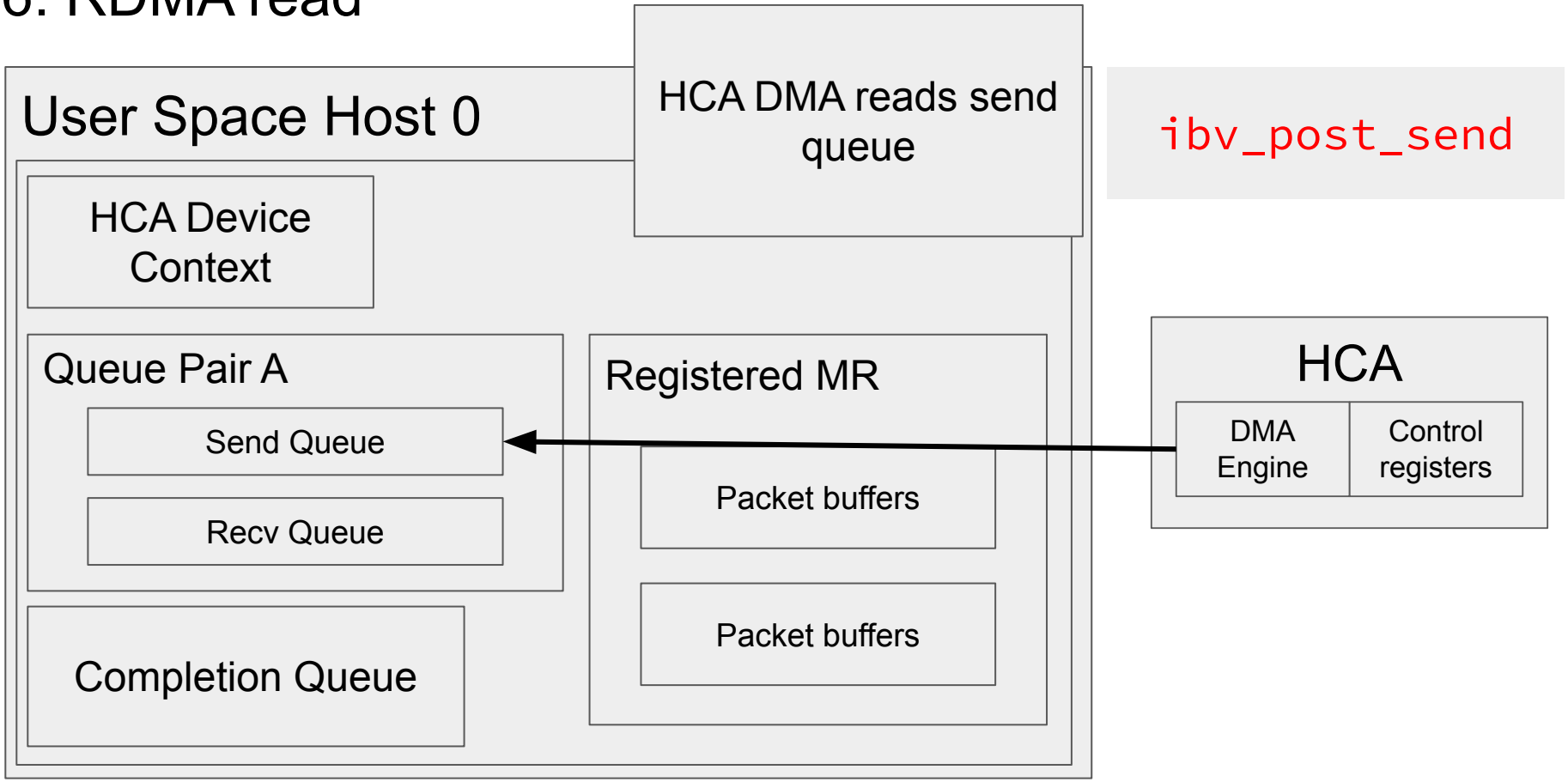
- rkey
- address of DMA buffer
- length

to the remote host over a separate channel like TCP

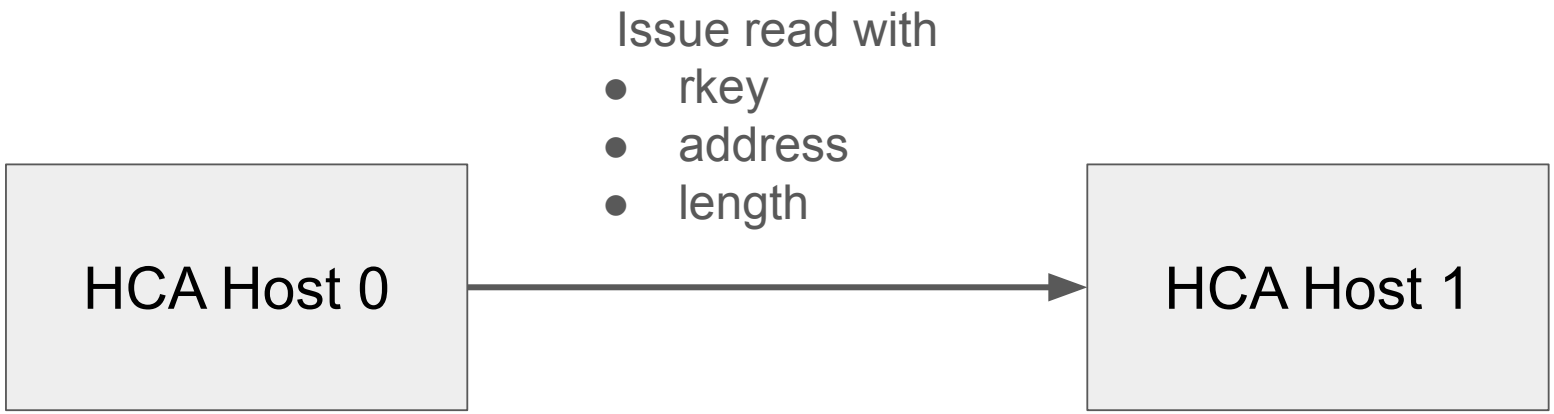
6. RDMA read


Submit a RDMA_READ work request to local QP
ibv_post_send
User Space Host 0
Protection Domain
HCA Device Context
Queue Pair A
Send Queue
Recv Queue
Registered MR
Packet buffers
Packet buffers
Completion Queue
HCA
DMA Engine
Control registers

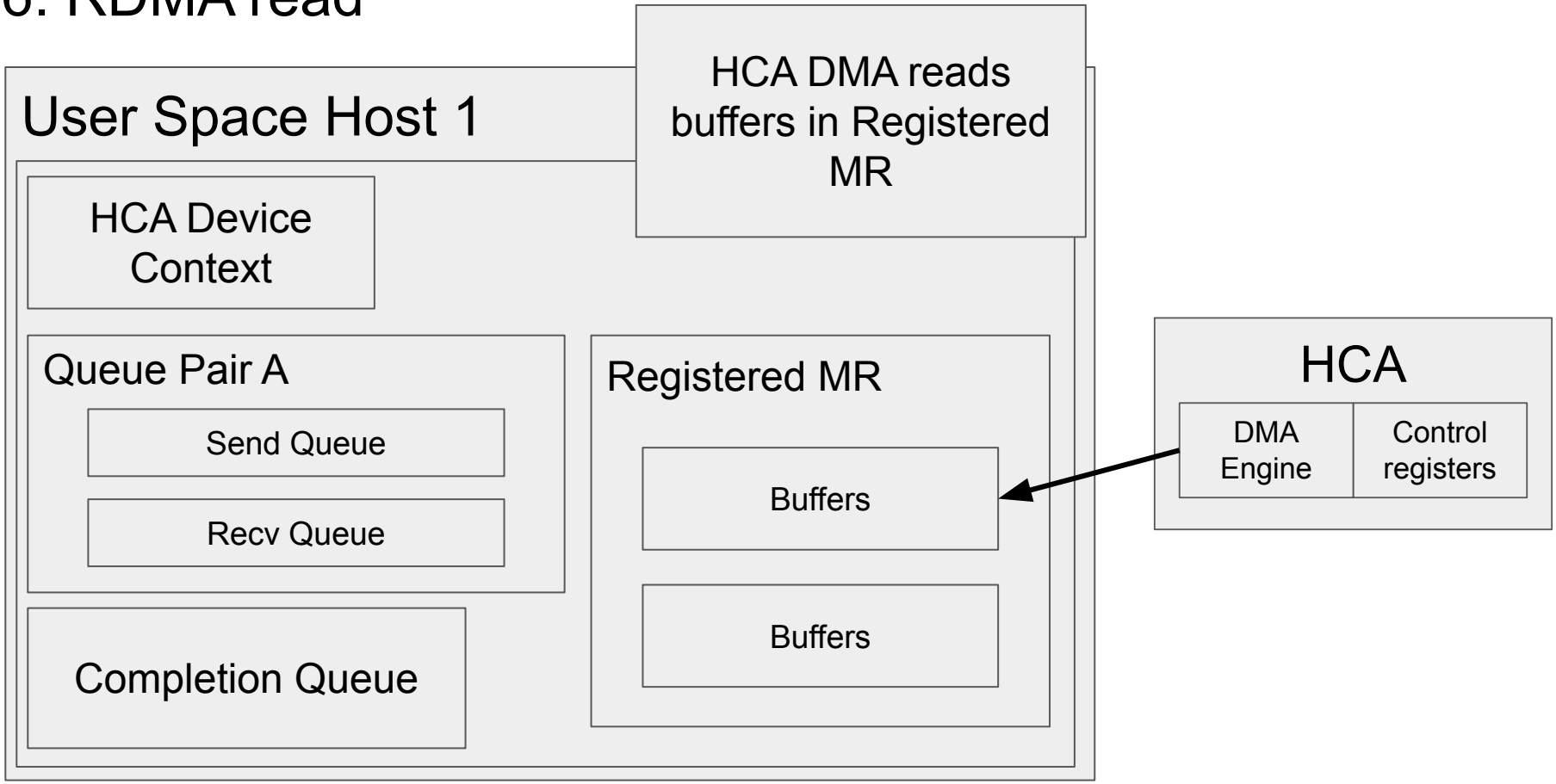
6. RDMA read


User Space Host 0
HCA DMA reads send queue
ibv_post_send
HCA Device Context
Queue Pair A
Send Queue
Recv Queue
Registered MR
Packet buffers
Packet buffers
Completion Queue
HCA
DMA Engine
Control registers

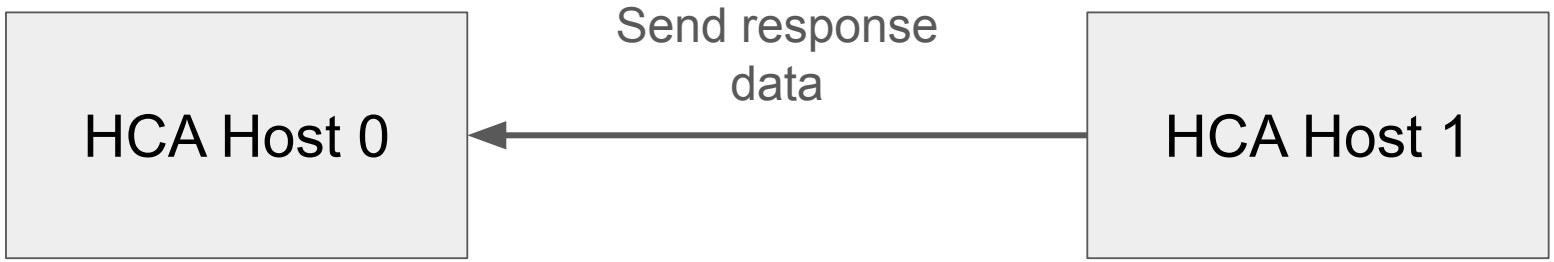
6. RDMA read



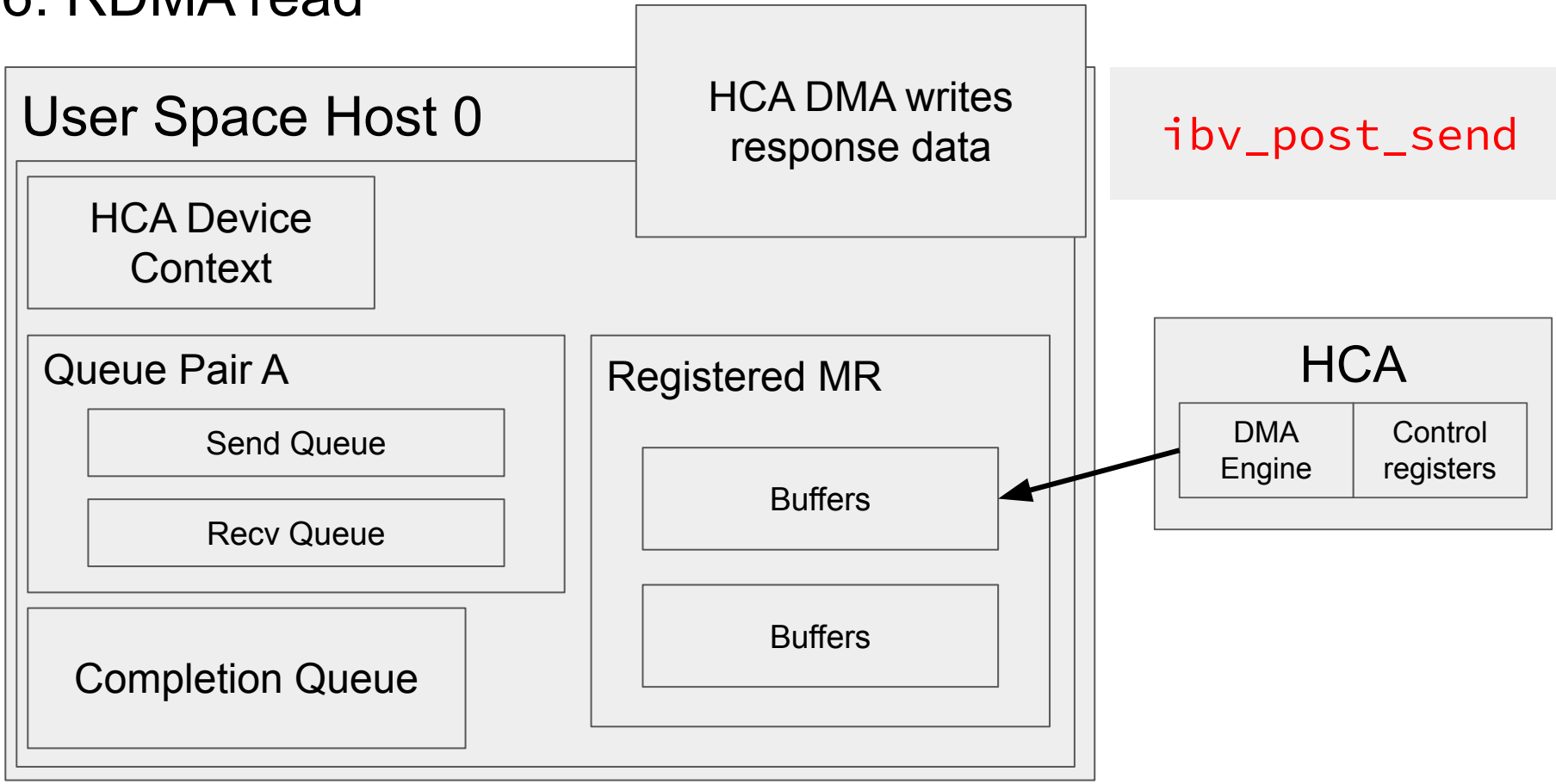
6. RDMA read


HCA DMA reads buffers in Registered MR
User Space Host 1
HCA Device Context
Queue Pair A
Send Queue
Recv Queue
Completion Queue
Registered MR
Buffers
Buffers
HCA
DMA Engine
Control registers

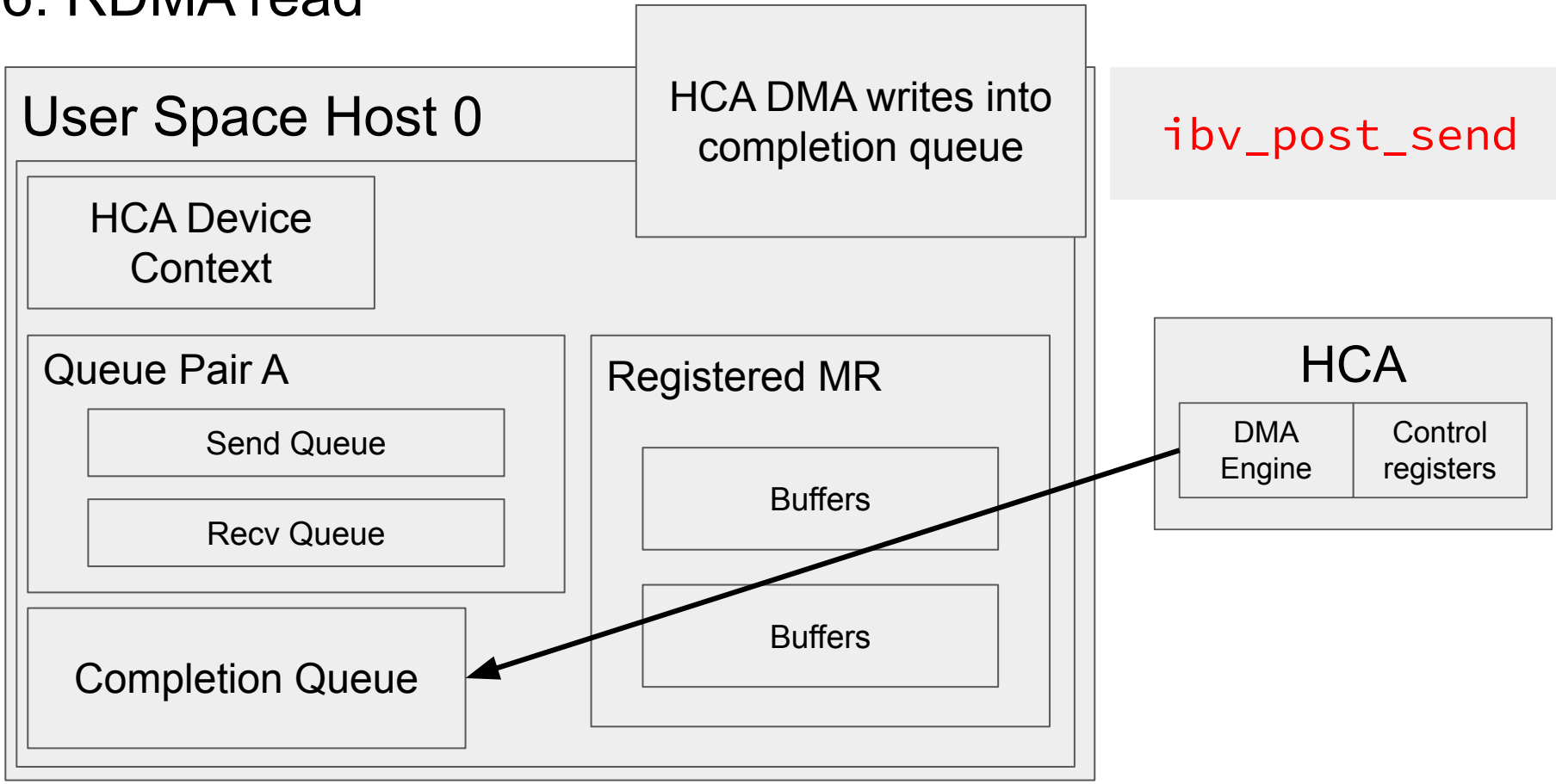
6. RDMA read



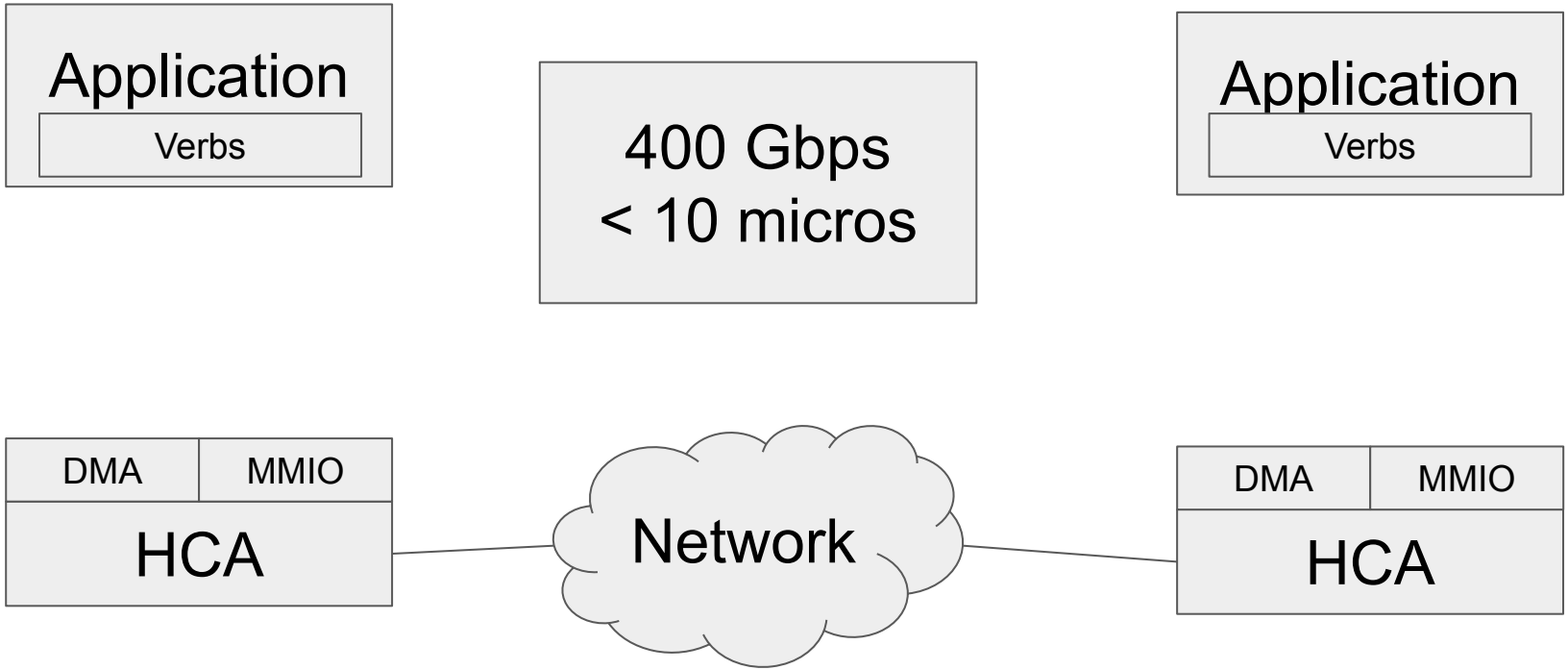
6. RDMA read

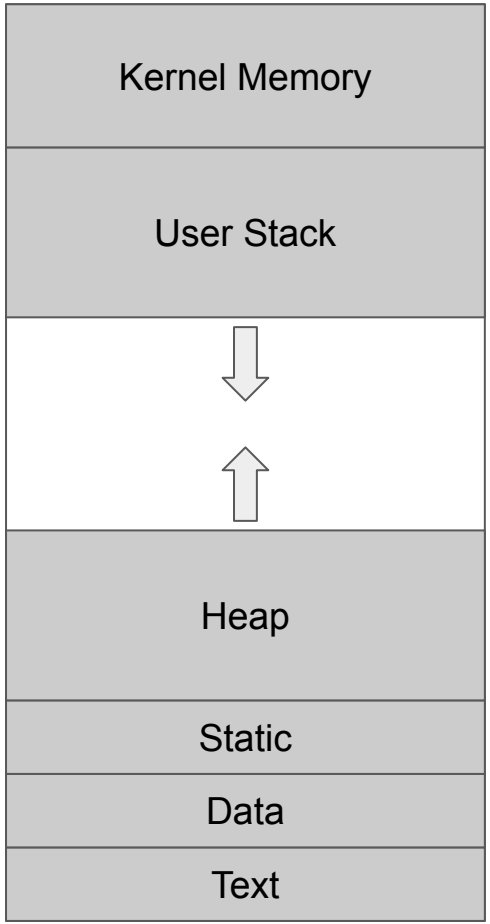

User Space Host 0
HCA DMA writes response data
ibv_post_send
HCA Device Context
Queue Pair A
Send Queue
Recv Queue
Registered MR
Buffers
Buffers
Completion Queue
HCA
DMA Engine
Control registers

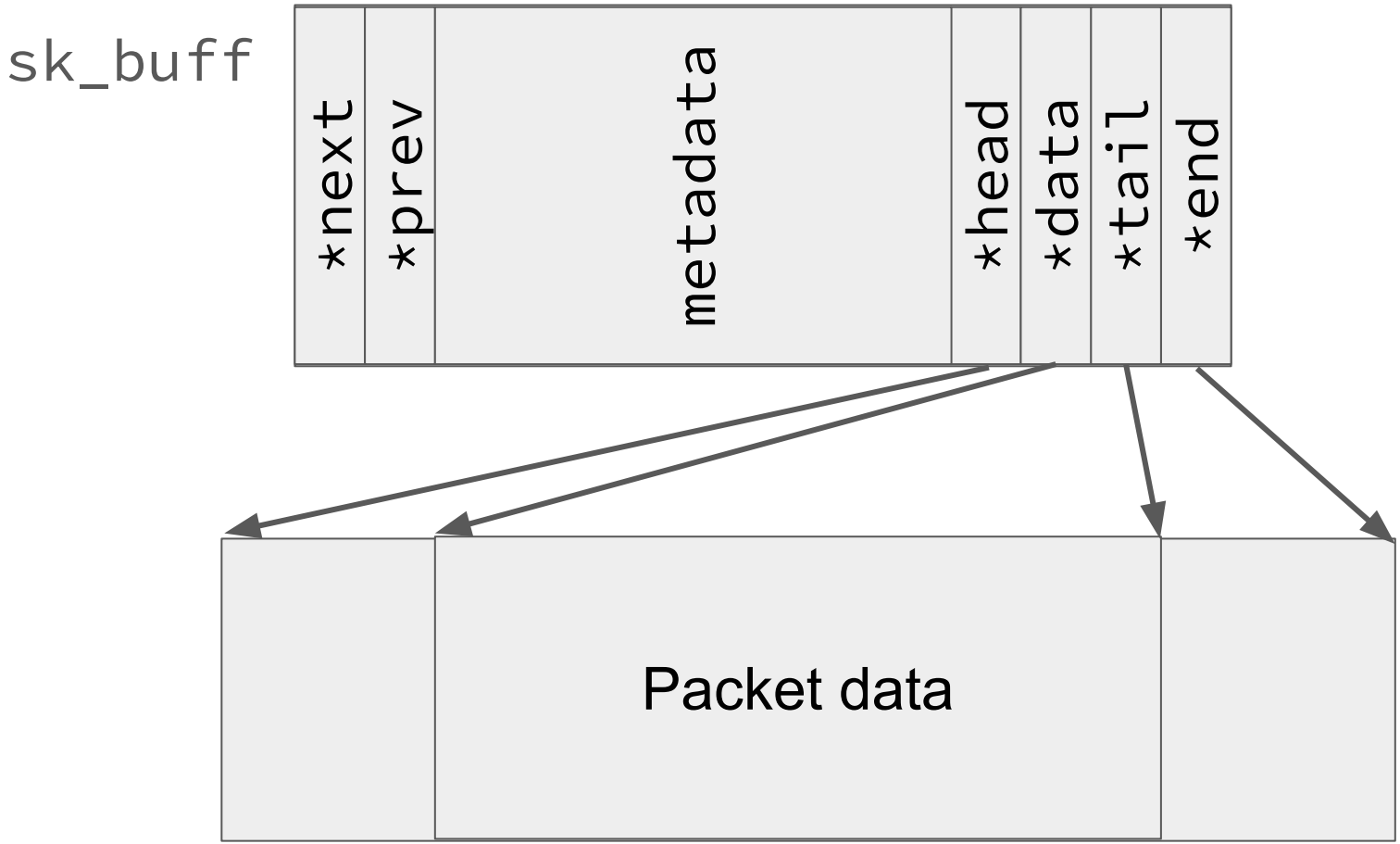
6. RDMA read


User Space Host 0
HCA DMA writes into completion queue
ibv_post_send
HCA Device Context
Queue Pair A
Send Queue
Recv Queue
Registered MR
Buffers
Buffers
Completion Queue
HCA
DMA Engine
Control registers

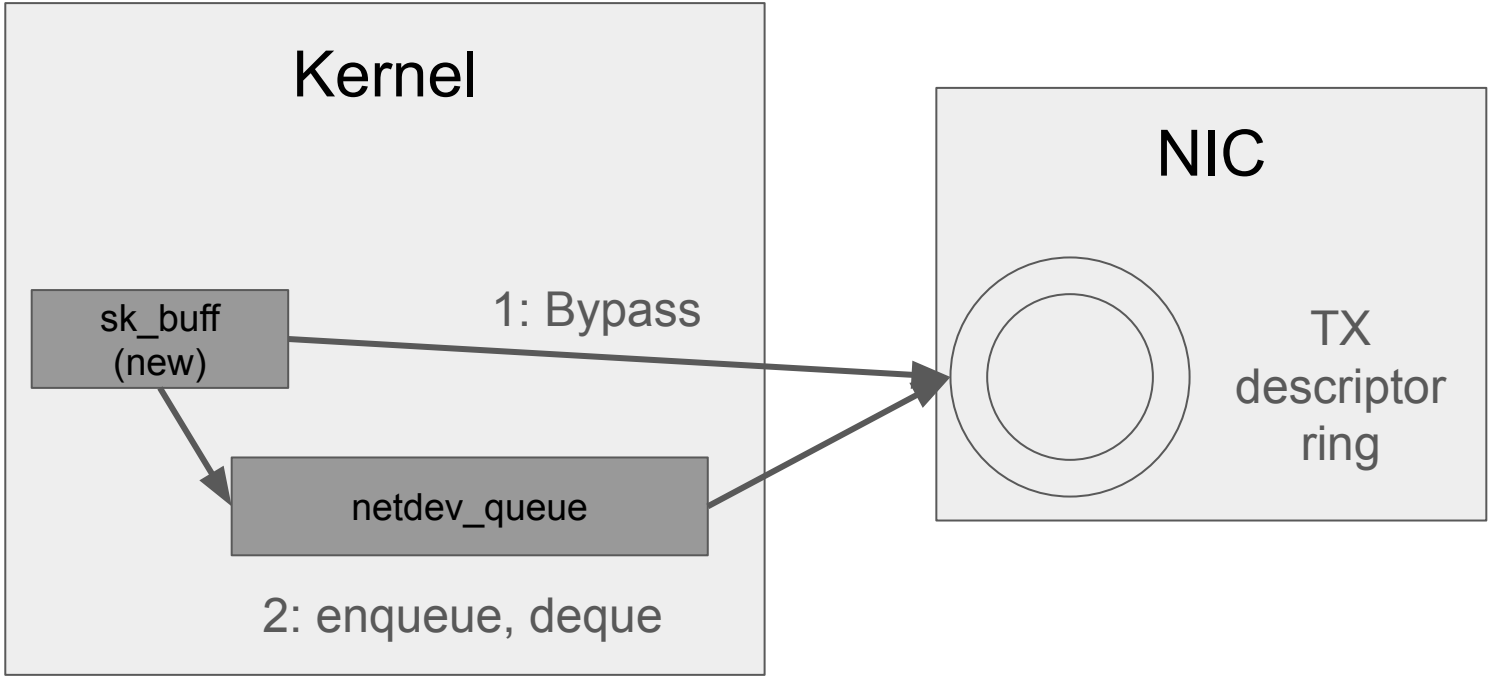
Infiniband RDMA

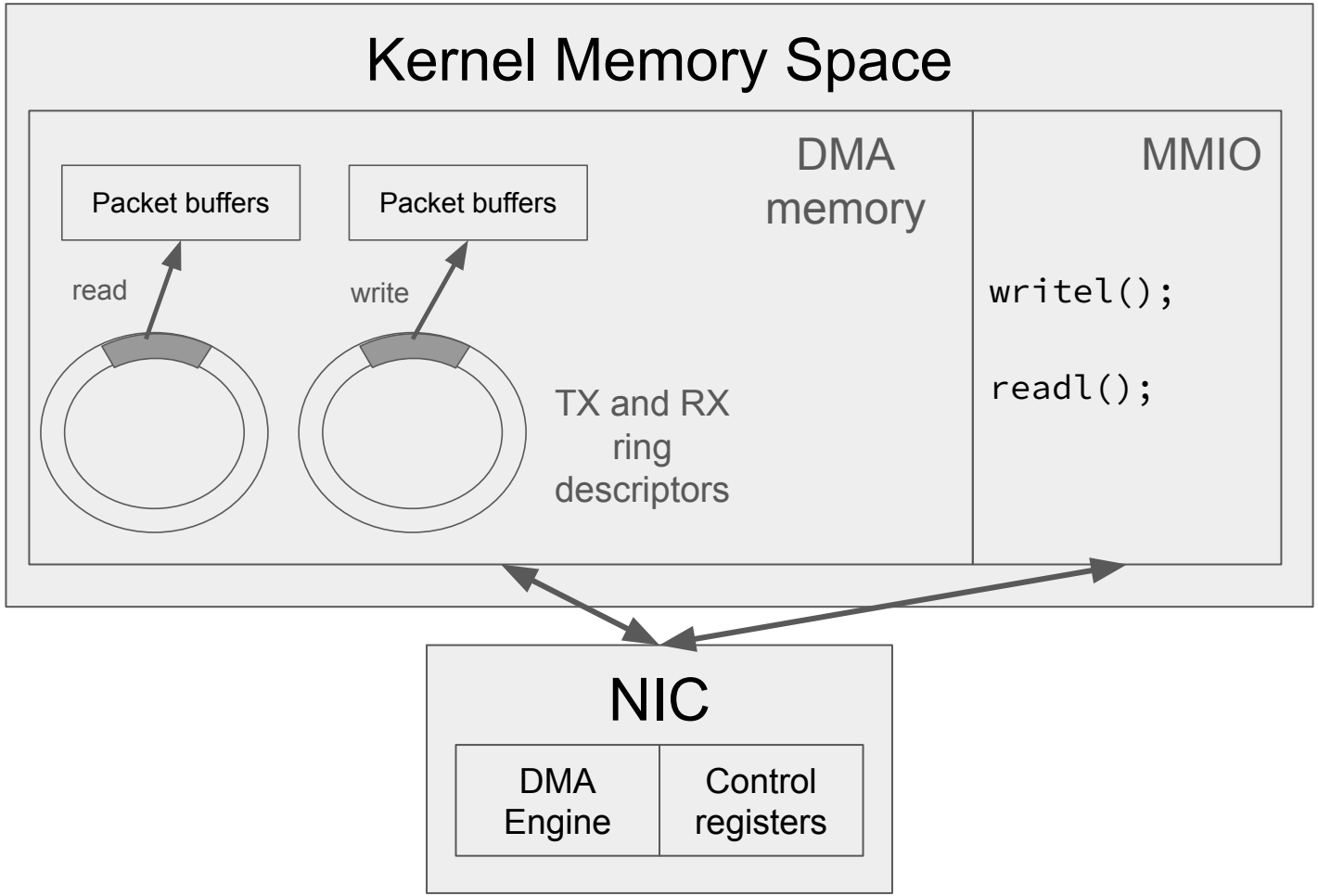


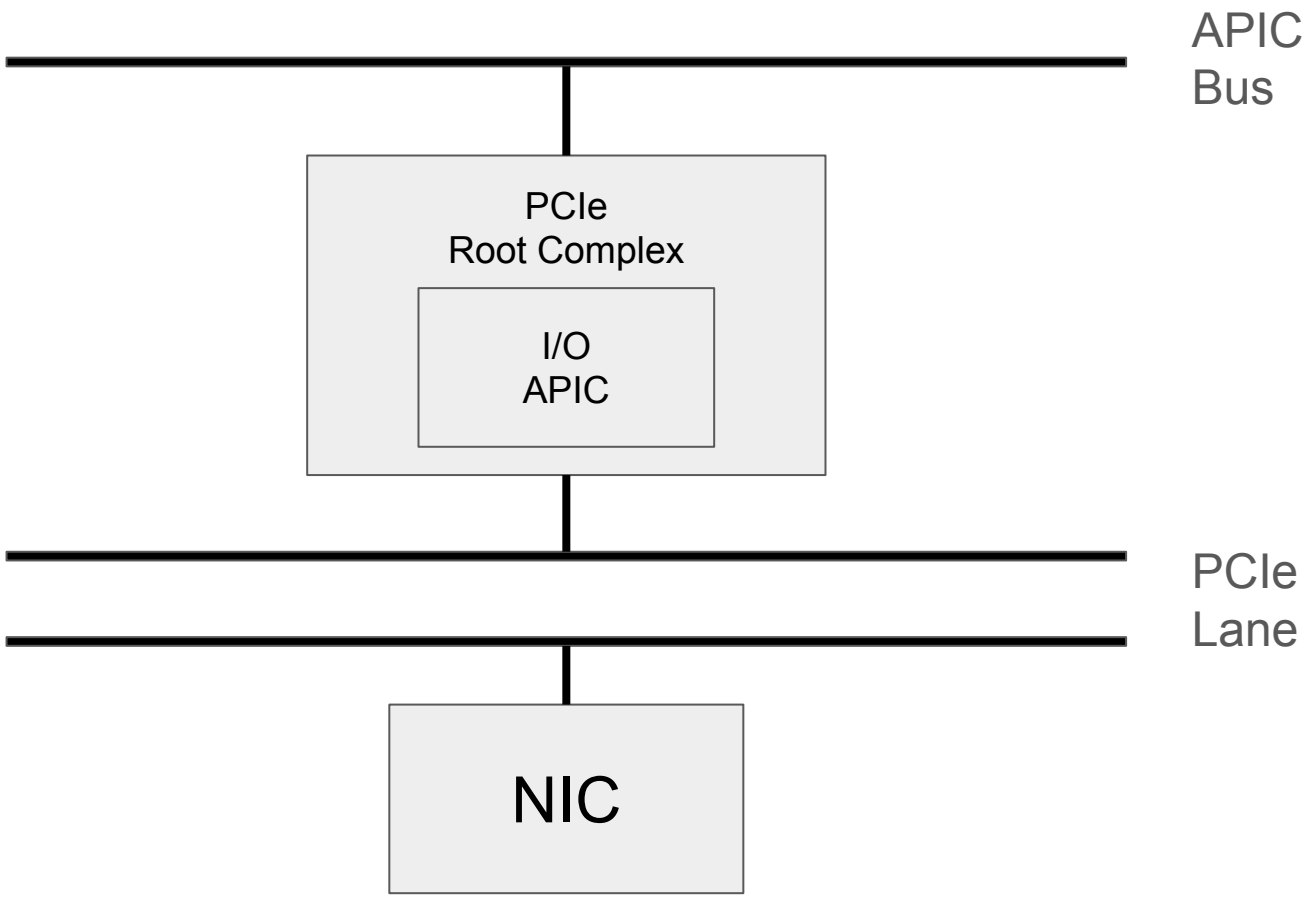

Kernel Memory
User Stack
Heap
Static
Data
Text

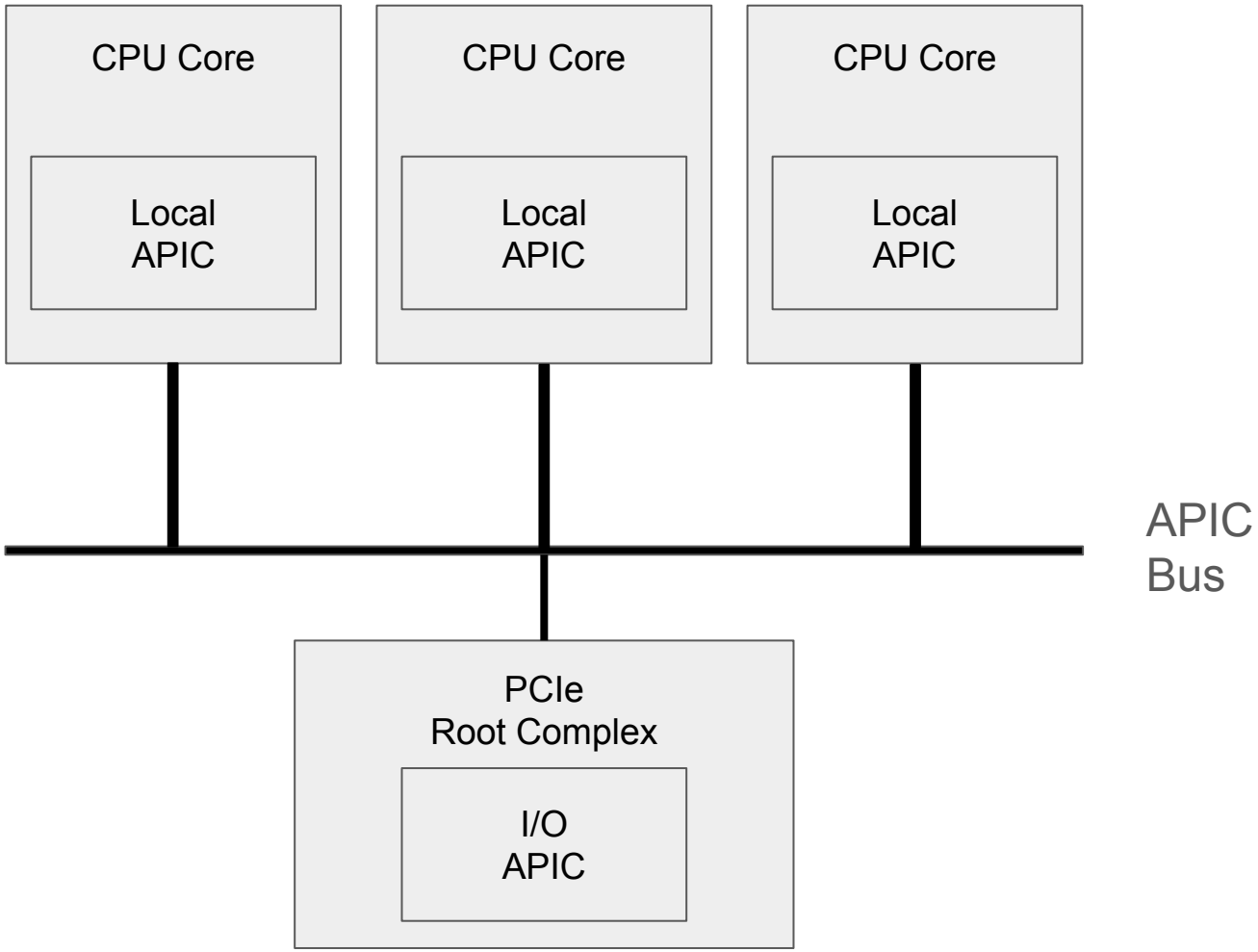

sk_buff
*next
*prev
metadata
*head
*data
*tail
*end
Packet data

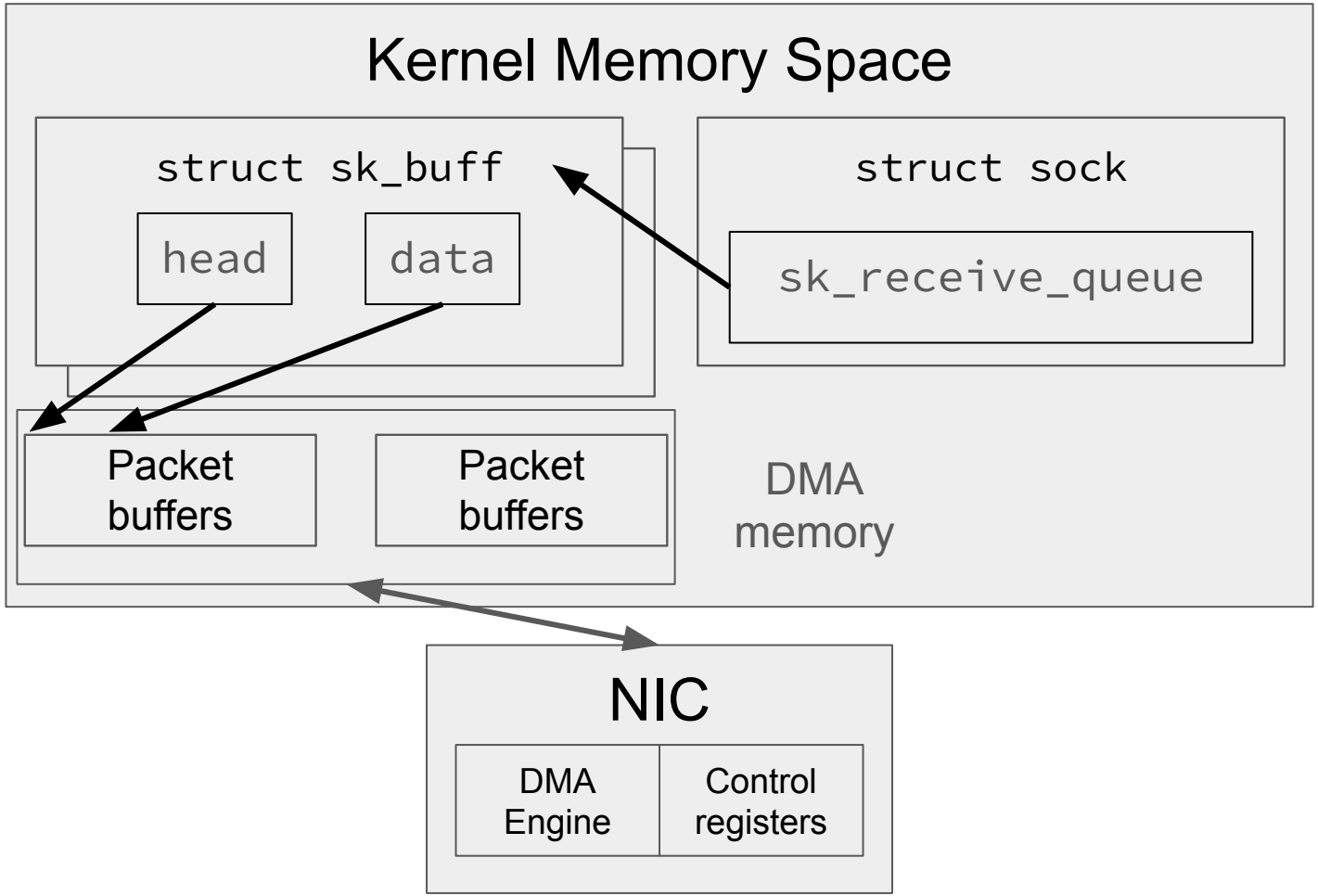
struct net_device
metadata (name of interface, flags, mac address)
operations on the network device
transmit queue
qdisc


Kernel
NIC
sk_buff
(new)
1: Bypass
TX
descriptor
ring
netdev_queue
2: enqueue, deque


Kernel Memory Space
DMA memory
MMIO
Packet buffers
Packet buffers
read
write
TX and RX ring descriptors
writel();
readl();
NIC
DMA Engine
Control registers


APIC
Bus
PCIe
Root Complex
I/O
APIC
PCIe
Lane
NIC


CPU Core
Local
APIC
CPU Core
Local
APIC
CPU Core
Local
APIC
APIC
Bus
PCIe
Root Complex
I/O
APIC


Kernel Memory Space
struct sk_buff
head
data
struct sock
sk_receive_queue
Packet buffers
Packet buffers
DMA memory
NIC
DMA Engine
Control registers